

Lecture 7

Binary Search Trees

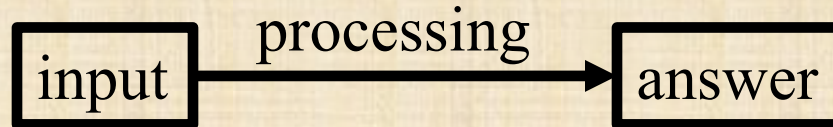


In previous lectures

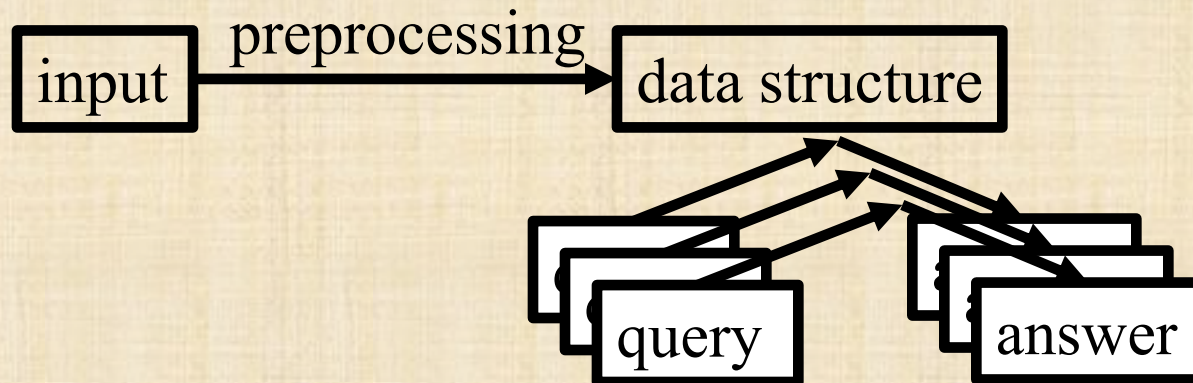
- ▷ We started with data structures for simple ADTs (sequence, stack, queue, priority queue)
- ▷ Then we discussed complexity analysis
- ▷ In the last 3 lectures we discussed sorting and selection algorithms (not data structures)

Algorithms vs. Data Structures

One-shot algorithm:



Data Structure:



back to the Sequence ADT

Implementation Operations			
Find			
Insert			
Delete			

Time Complexity

Array & Linked list

Implementation Operations	Sorted Array	Sorted Linked List	Unsorted array
Find			
Insert			
Delete			

Time Complexity Array & Linked list

Implementation Operations	Sorted Array	Sorted Linked List	Unsorted array
Find	$O(\log n)$	$O(n)$	$O(n)$
Insert	$O(n)$	$O(n)$	$O(1)$
Delete	$O(n)$	$O(n)$	$O(n)$

Can we do better?

Here is our wish list

Implementation Operations	Sorted Array	Sorted Linked List	Unsorted array	Desired
Find	$O(\log n)$	$O(n)$	$O(n)$	$O(1)$
Insert	$O(n)$	$O(n)$	$O(1)$	$O(1)$
Delete	$O(n)$	$O(n)$	$O(n)$	$O(1)$

But we cannot achieve that

Implementation Operations	Sorted Array	Sorted Linked List	Unsorted array	Desired
Find	$O(\log n)$	$O(n)$	$O(n)$	$O(1)$
Insert	$O(n)$	$O(n)$	$O(1)$	$O(1)$
Delete	$O(n)$	$O(n)$	$O(n)$	$O(1)$

Here is what we can achieve

Implementation Operations	Sorted Array	Sorted Linked List	Unsorted array	Binary Search Tree
Find	$O(\log n)$	$O(n)$	$O(n)$	$O(\log n)$
Insert	$O(n)$	$O(n)$	$O(1)$	$O(\log n)$
Delete	$O(n)$	$O(n)$	$O(n)$	$O(\log n)$

Reminder: Recursive Definition of a Tree

A tree is a set of nodes which is either

▷ empty

or

▷ has one node called the **root** which points to the roots of zero or more mutually **disjoint** trees (called subtrees)

disjoint: no shared nodes

Recursive Definition of a Binary Tree

A binary tree is a set of nodes which is either

- ▷ empty

or

- ▷ has one node called the **root** which points to a subset of:

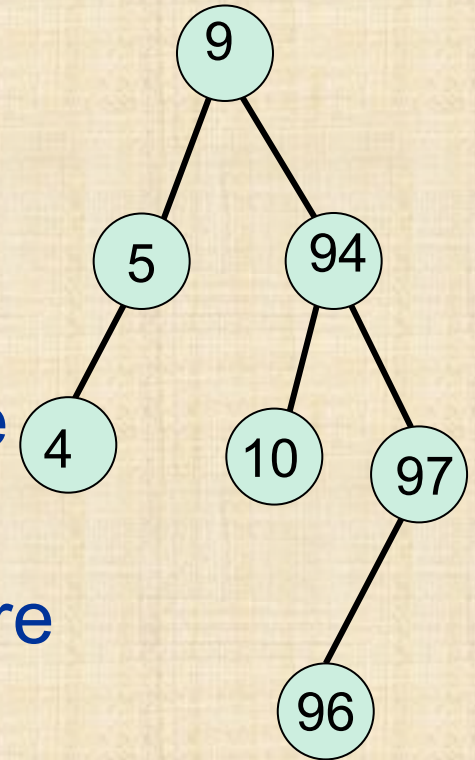
- ▷ {Left Binary tree T_L , Right Binary tree T_R };

- Where T_L and T_R **are disjoint**: no shared nodes

- ▷ T_L and T_R are called the left and right subtrees of the root.

Binary Search Trees (BST)

- ▷ A binary tree with an additional restriction on the keys:
- ▷ BST property: for every node v
 - ▷ The keys in the v 's left subtree are smaller than the key of v
 - ▷ The keys in the v 's right subtree are larger than the key of v

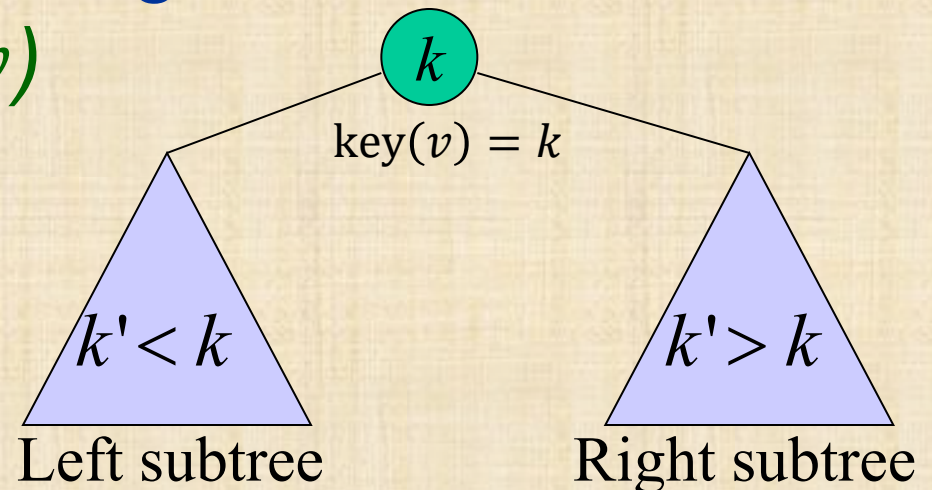


(Assume for now that all keys are different)

Binary Search Tree

Let v be a node in a binary search tree

- ▷ If y is a node in the left subtree then
 $key(y) < key(v)$
- ▷ If y is a node in the right subtree then
 $key(y) > key(v)$



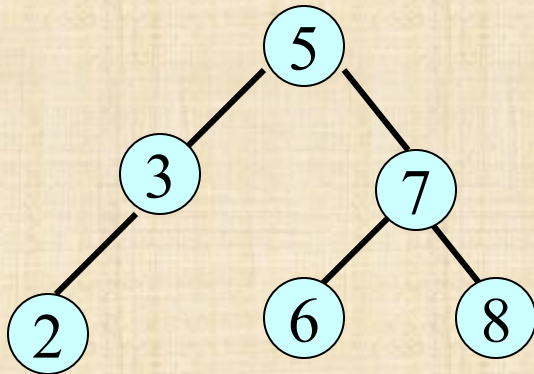
BST - Depth

▷ Maximum depth:

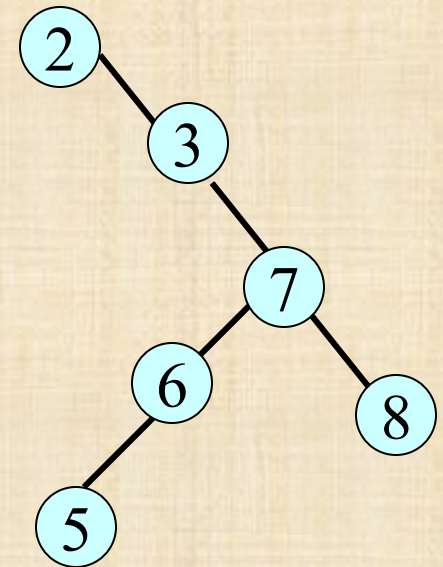
▷ $O(n)$

▷ Minimum depth:

▷ $\Omega(\log n)$

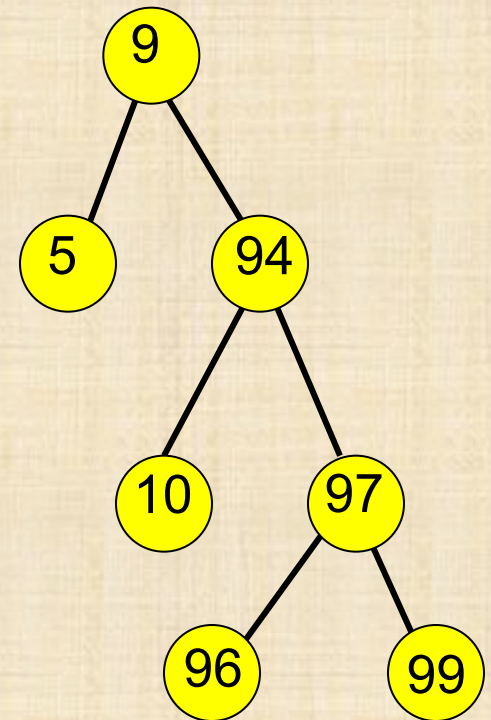


Two binary search trees
with the same set of keys

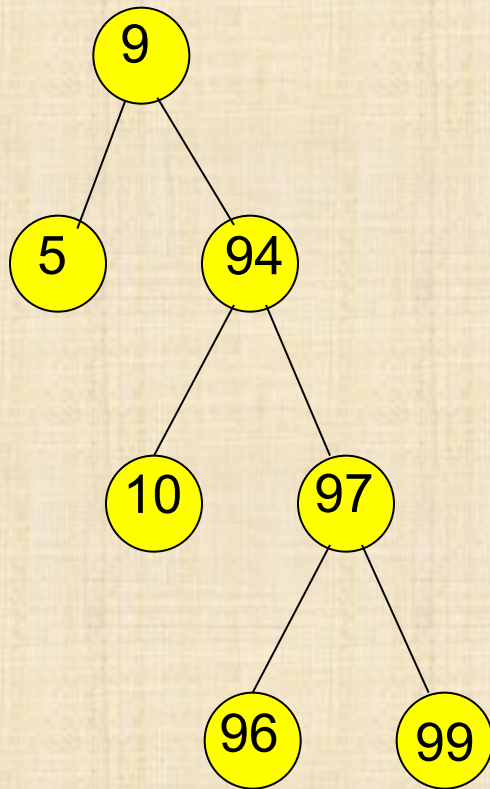


Operations on BST

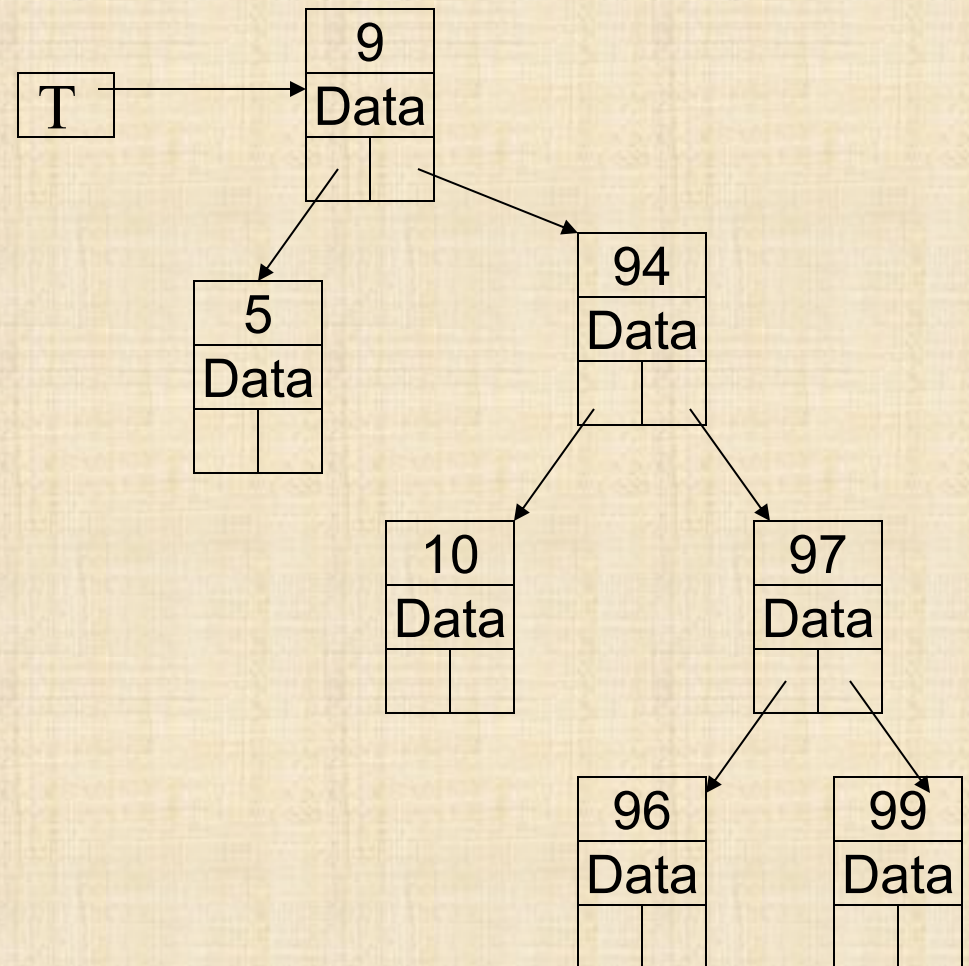
- ▷ *Find*
- ▷ *Insert*
- ▷ *Delete*
- ▷ *Initialize*
- ▷ *Rank*
- ▷ *FindMin*
- ▷ *FindMax*
- ▷ *Successor*
- ▷ *Predecessor*
- ▷ *Split*
- ▷ *Join*



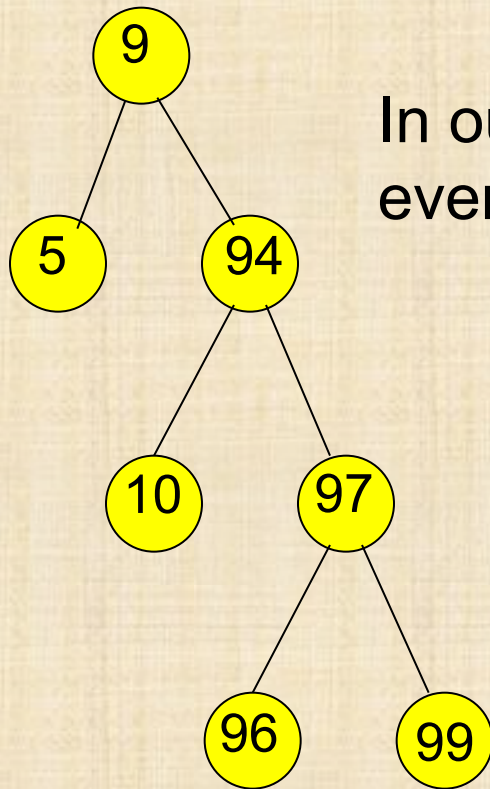
Implementation: BST



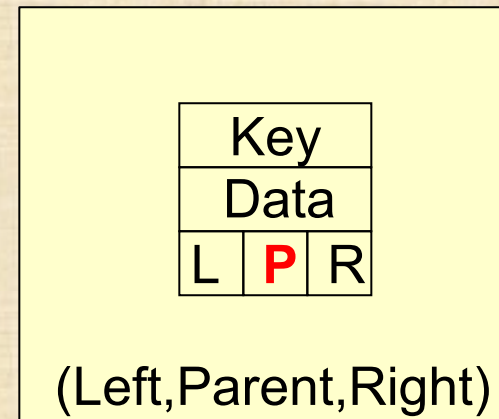
key	
Data	
L	R



Implementation: BST (with parent pointers)



In our implementation it will be useful that every node also stores a pointer to its parent



Find

Find(T, x) :

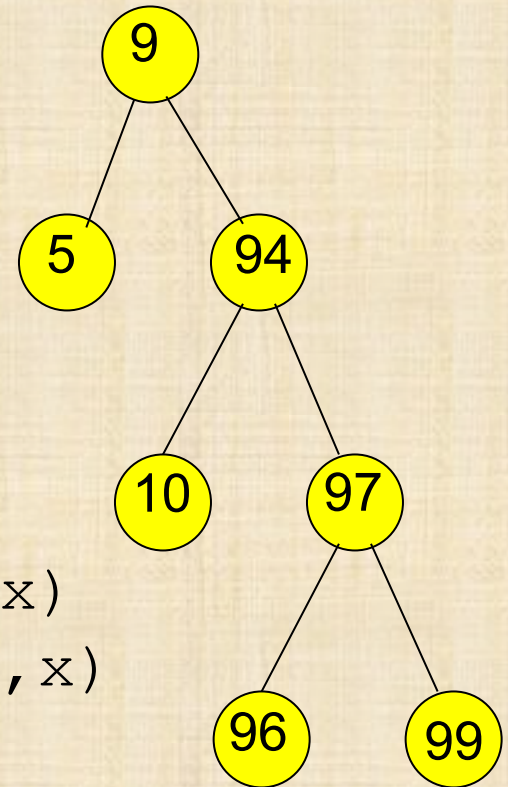
case

T = null : **return** null

T.key = x : **return** T

T.key > x : **return** *Find*(T.left, x)

T.key < x : **return** *Find*(T.right, x)



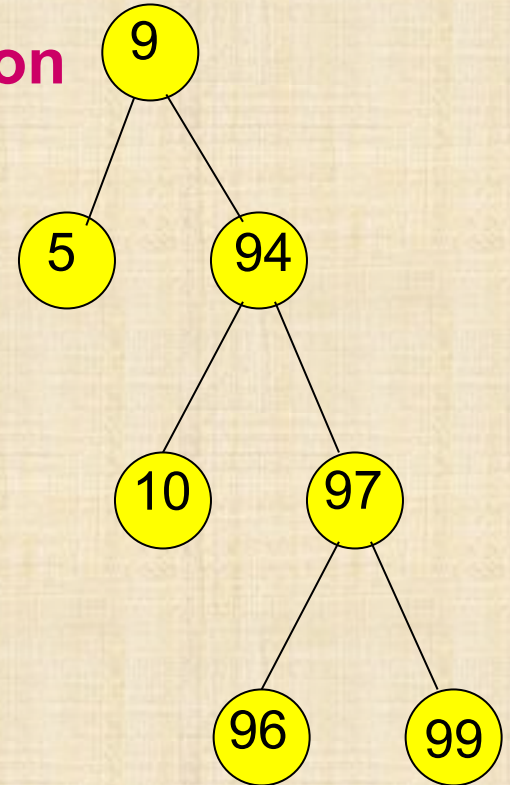
Time complexity?

T: the root of a BST
x: key value

Find

non-recursive implementation

```
Find(T, x) :  
while T≠null and T.key≠x:  
    if x < T.key: T = T.left  
    else T = T.right  
return T
```



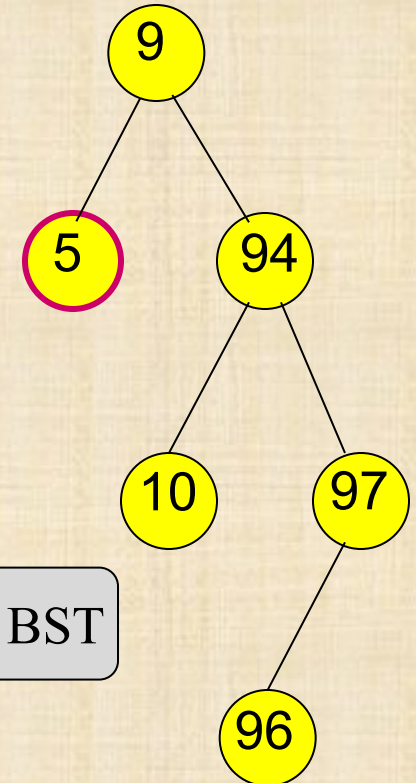
Time complexity?

T: the root of a BST
x: key value

FindMin

```
FindMin(T) :  
if T.left = null  
    then return T  
else return FindMin(T.left)
```

Time complexity?



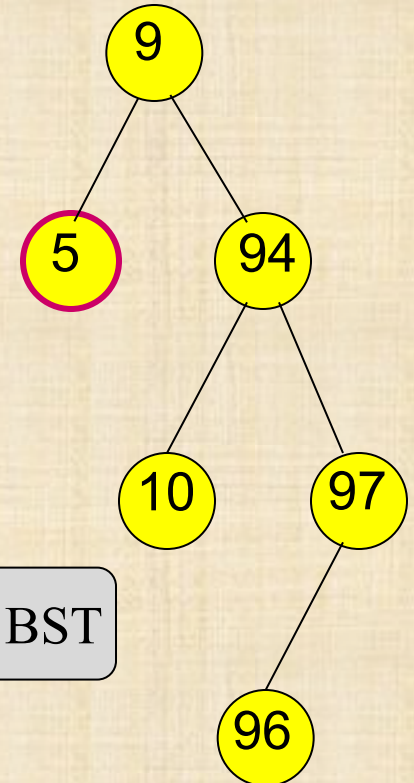
T: the root of a BST

FindMin

non-recursive implementation

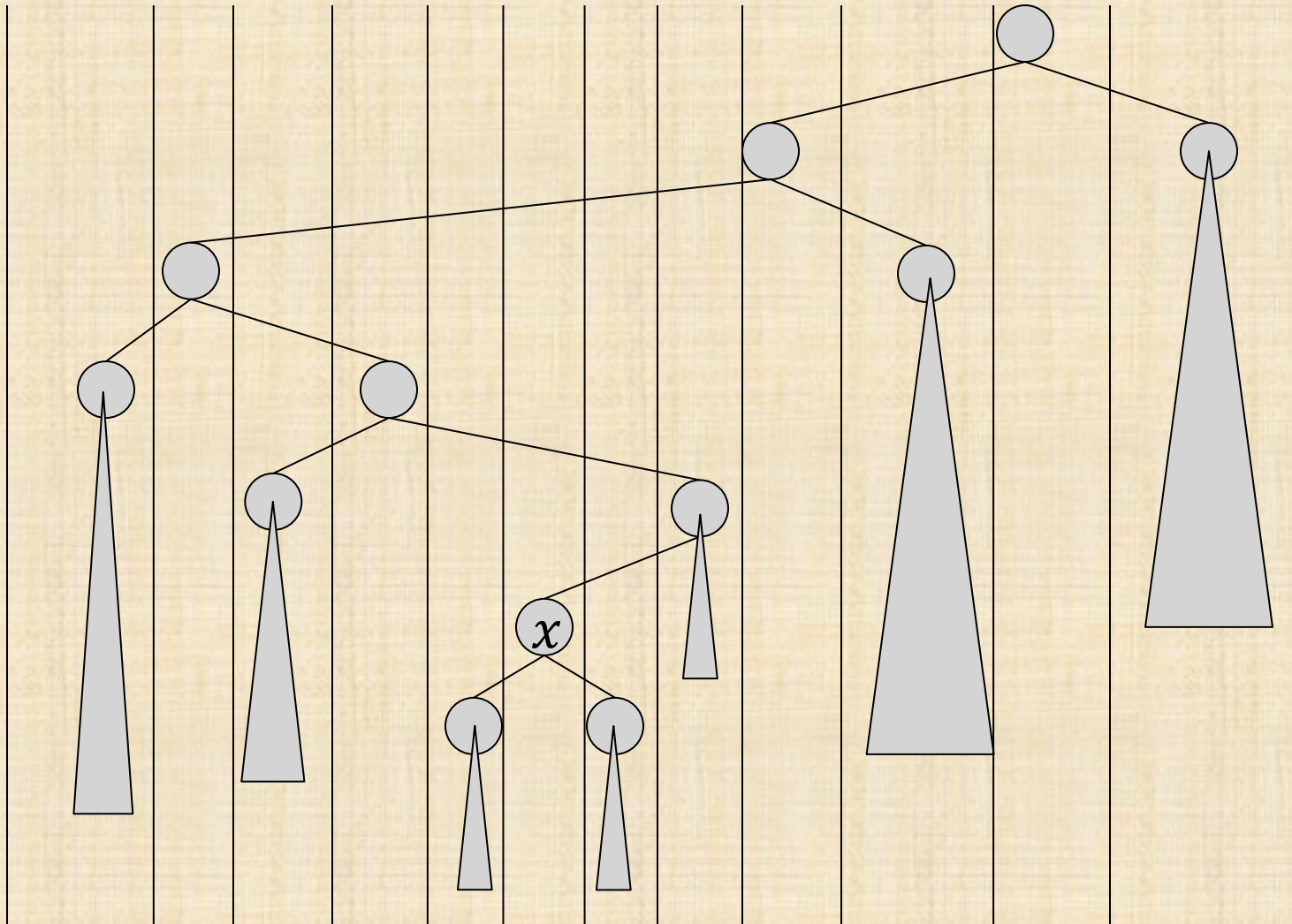
```
FindMin(T) :  
while T.left  $\neq$  null  
    do T = T.left  
return T
```

Time complexity?

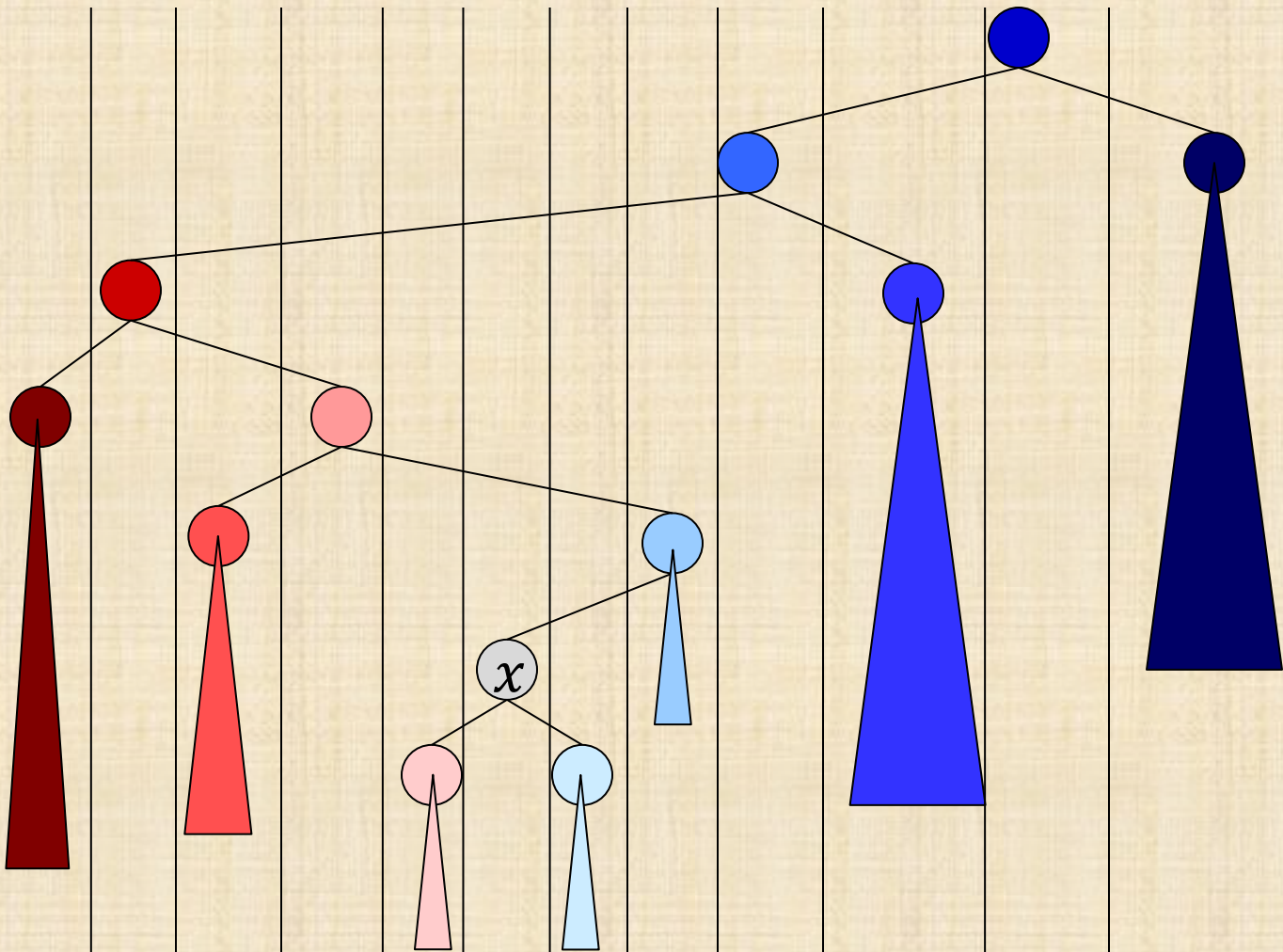


T: the root of a BST

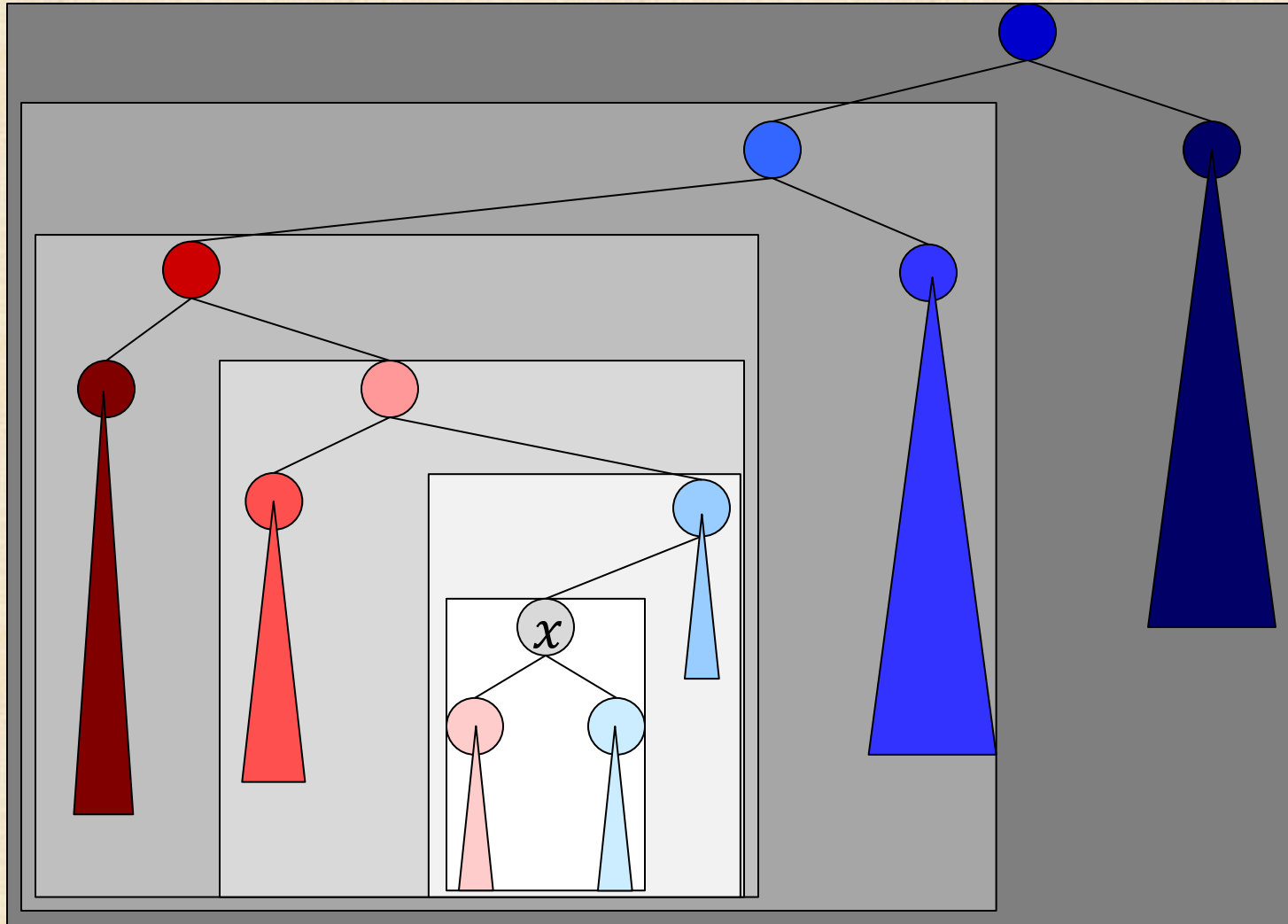
Here is a BST



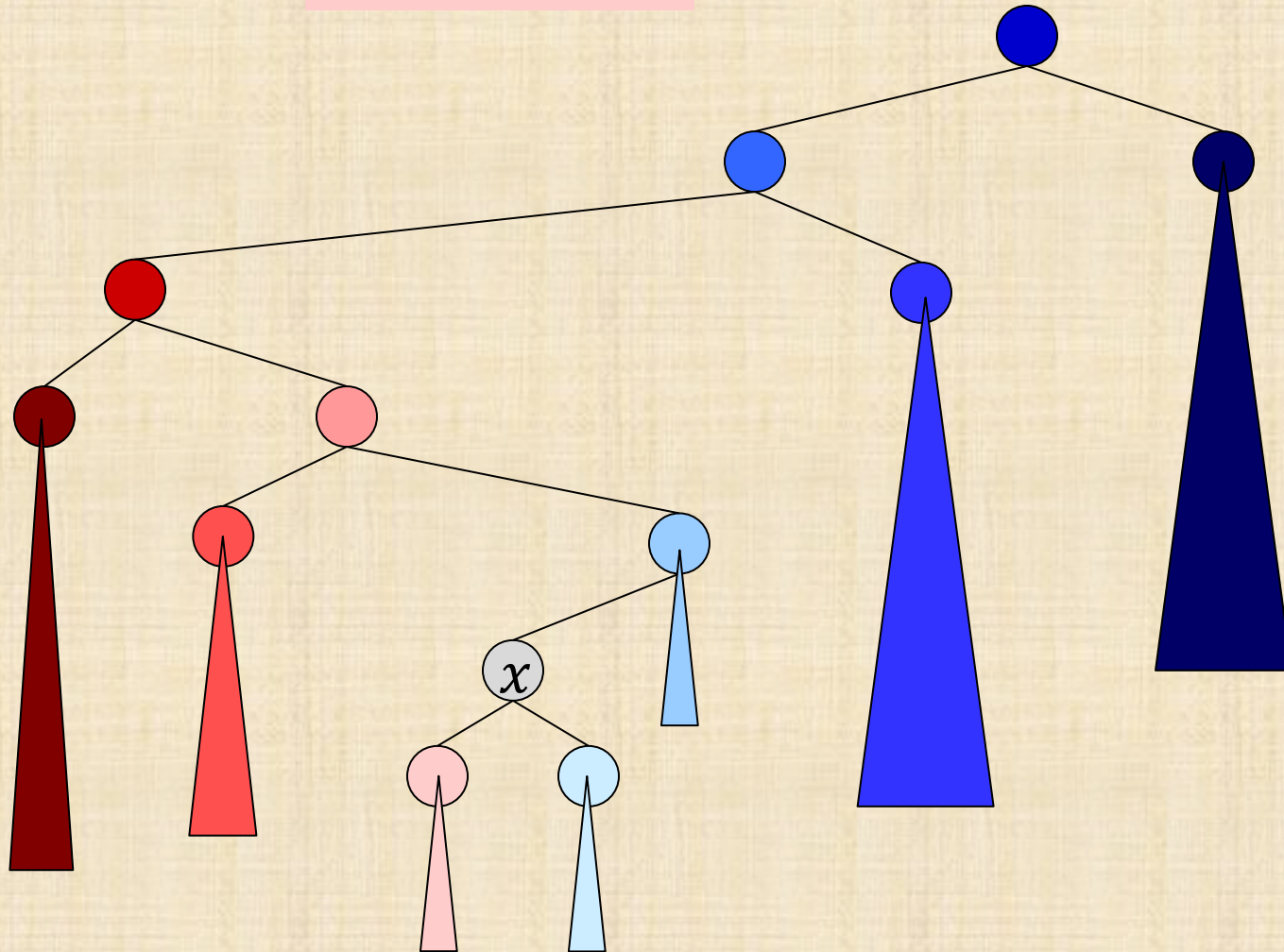
Here is a BST



Here is a BST



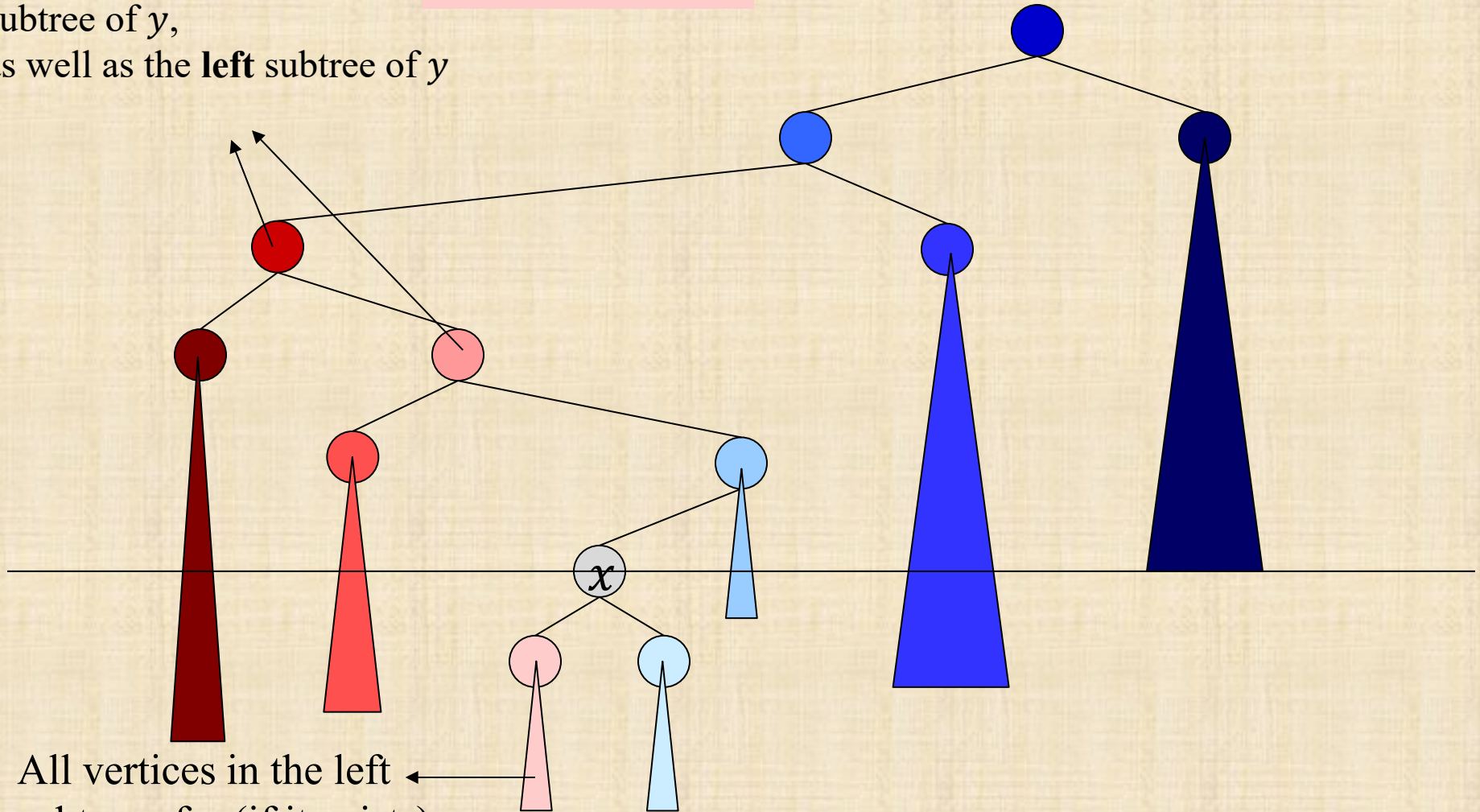
Where are the elements smaller than x



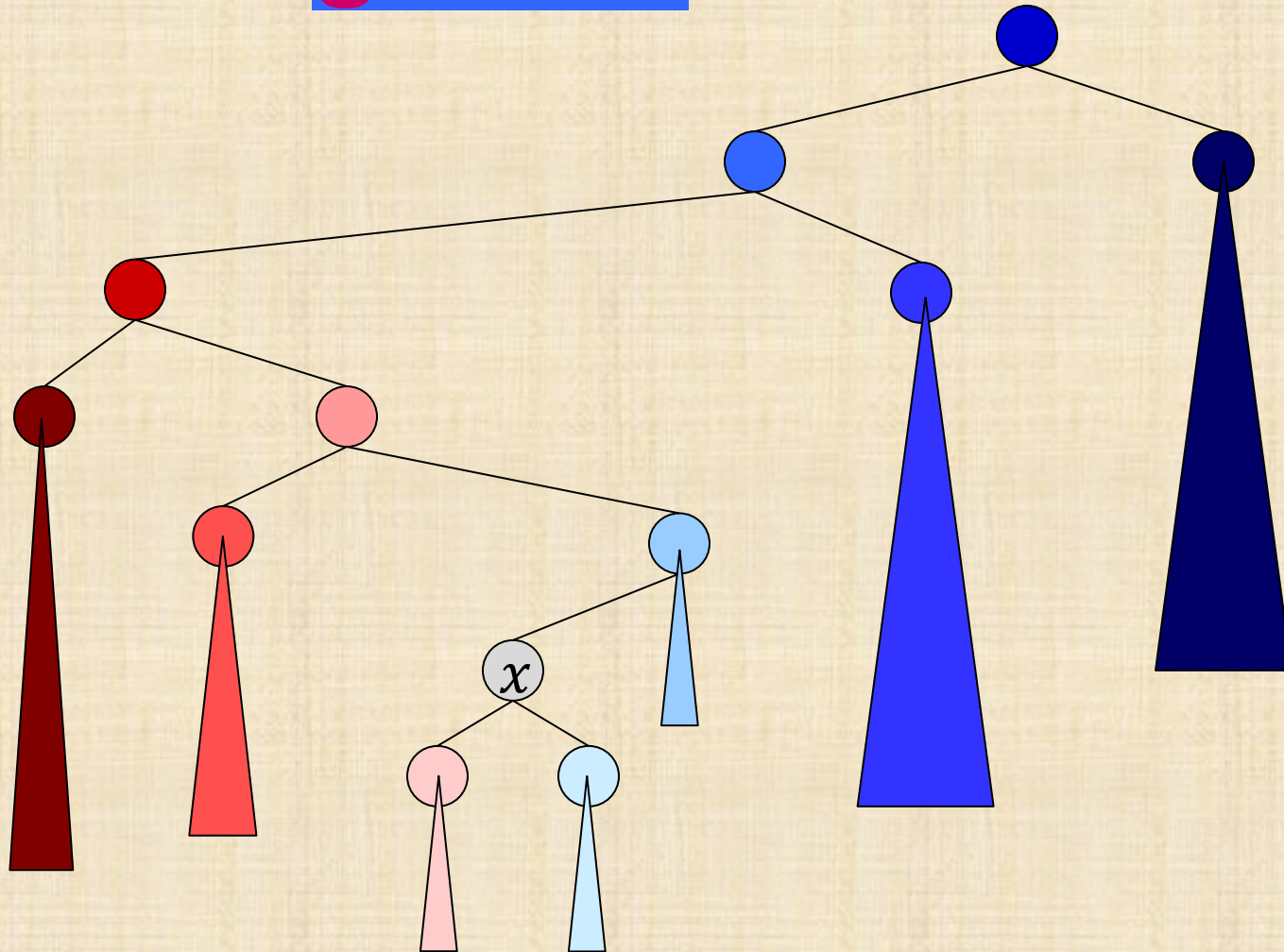
Where are the elements

smaller than x

Any ancestor y of x
such that, x is in the **right**
subtree of y ,
as well as the **left** subtree of y



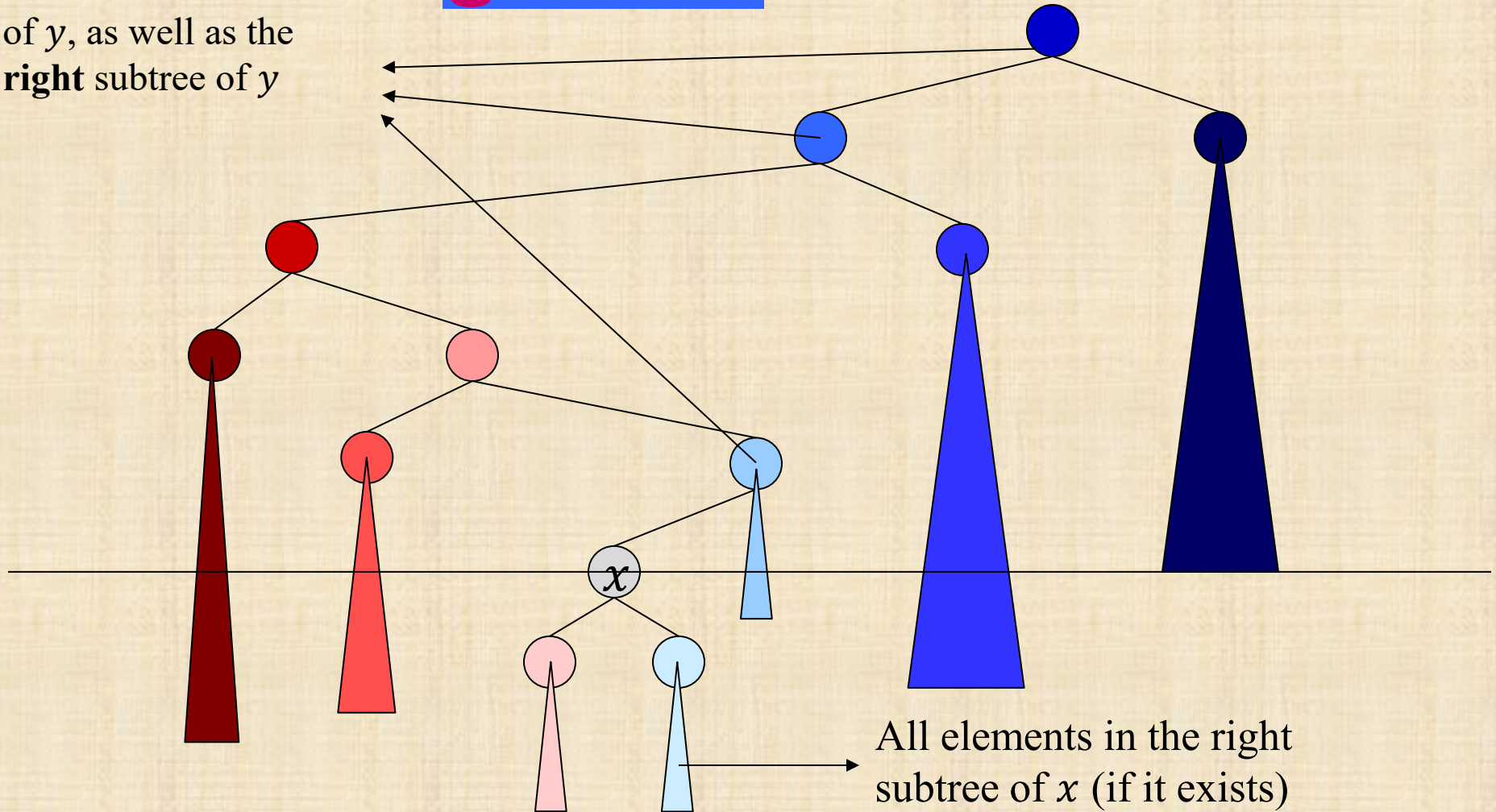
Where are the elements greater than x



Where are the elements

greater than x

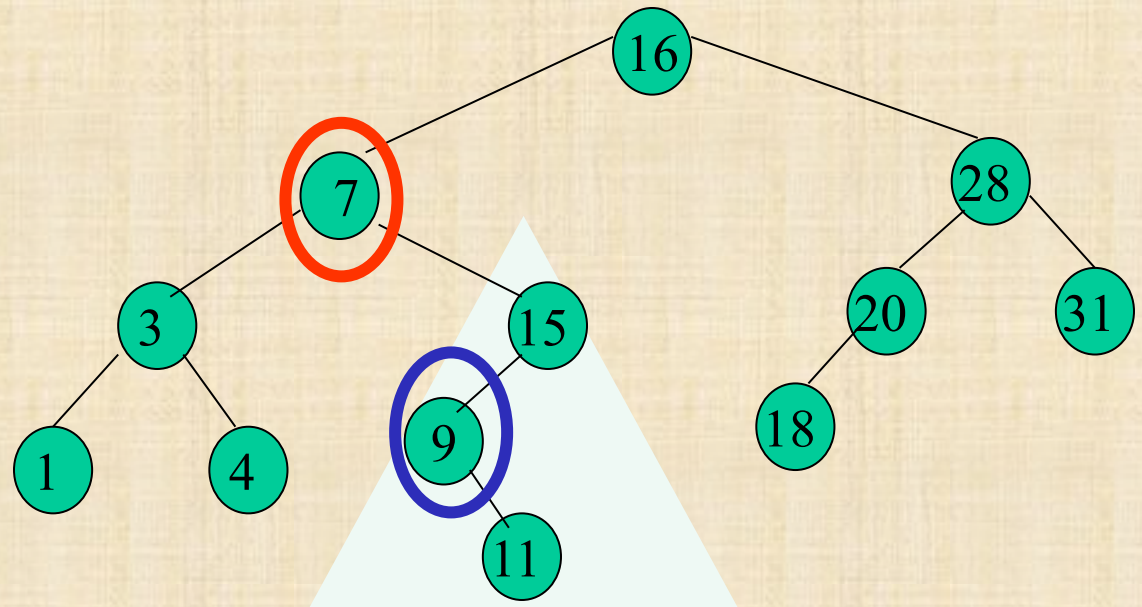
Any ancestor y of x such that x is in the **left** subtree of y , as well as the **right** subtree of y



Successor of x

- ▷ If x has a right sub-tree then the successor of x is the minimum value in this sub-tree.

Example: $\text{successor}(7)=9$



Successor of x

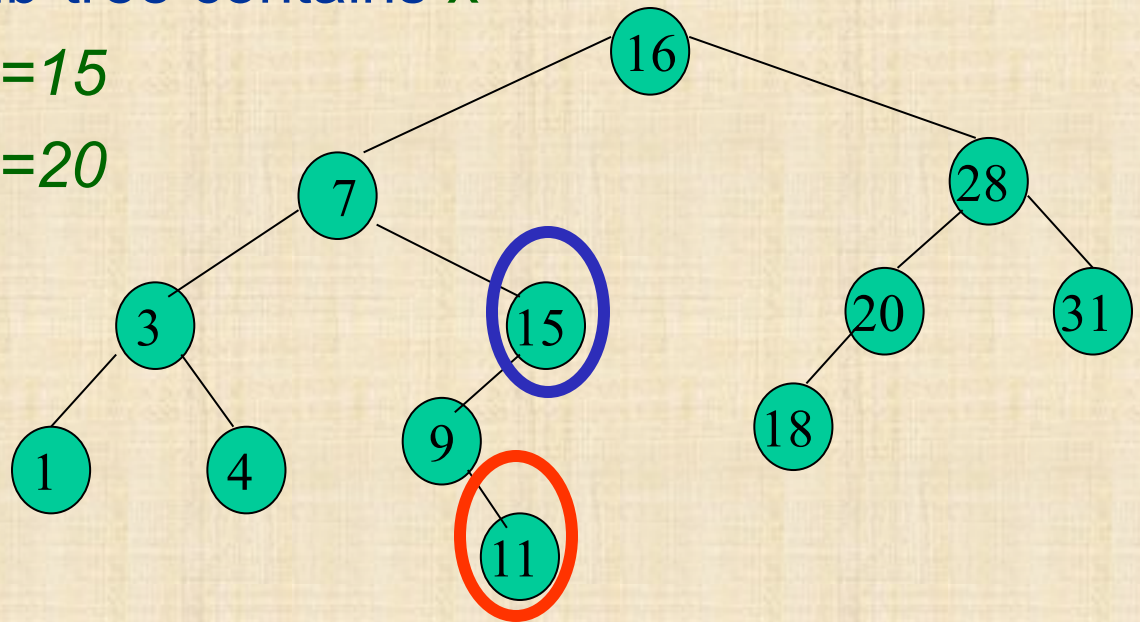
- ▷ If x has a right sub-tree then the successor of x is the minimum value in this sub-tree.

Example: $\text{successor}(7)=9$

- ▷ Otherwise, the successor of x is the lowest ancestor of x whose left sub-tree contains x

- ▷ $\text{successor}(11)=15$

- ▷ $\text{successor}(18)=20$



Successor of x

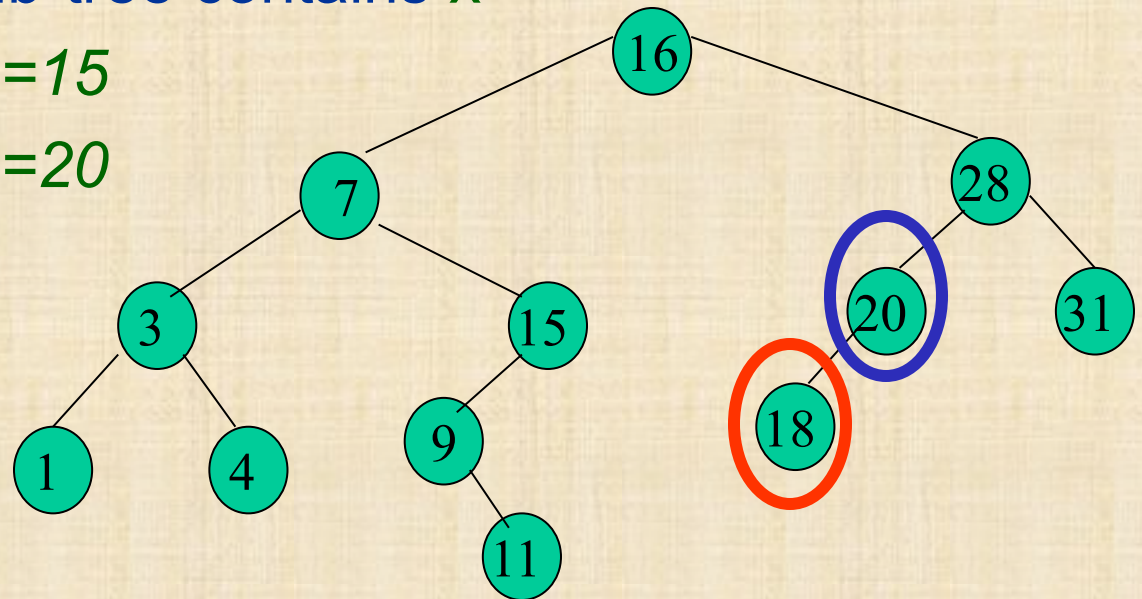
- ▷ If x has a right sub-tree then the successor of x is the minimum value in this sub-tree.

Example: $\text{successor}(7)=9$

- ▷ Otherwise, the successor of x is the lowest ancestor of x whose left sub-tree contains x

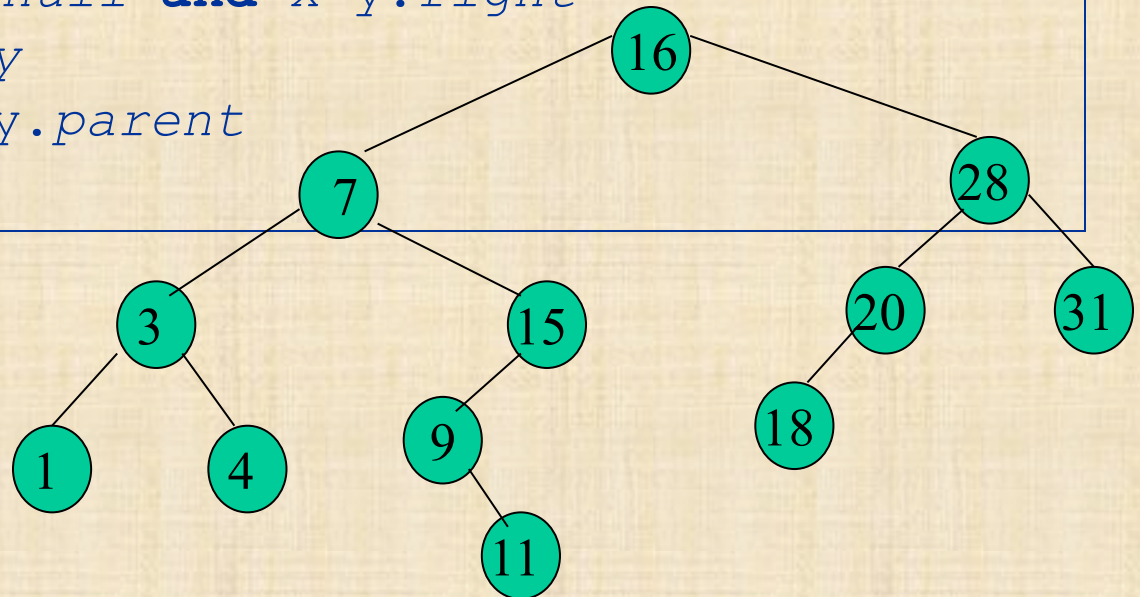
- ▷ $\text{successor}(11)=15$

- ▷ $\text{successor}(18)=20$



Successor

```
Tree-Successor(x)
  if x.right ≠ null
    then
      return FindMin(x.right)
  else
    y ← x.parent
    while y ≠ null and x=y.right
      x ← y
      y ← y.parent
    return y
```

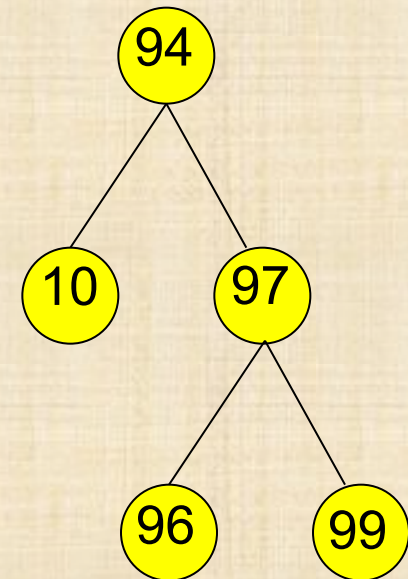


Time complexity?

Insert

Insert(T, x)

- ▷ Do a “Find” operation for x
- ▷ If x is found → announce key exists
- ▷ Else, “Find” stops at a *null* pointer
- ▷ Insert Node with x there



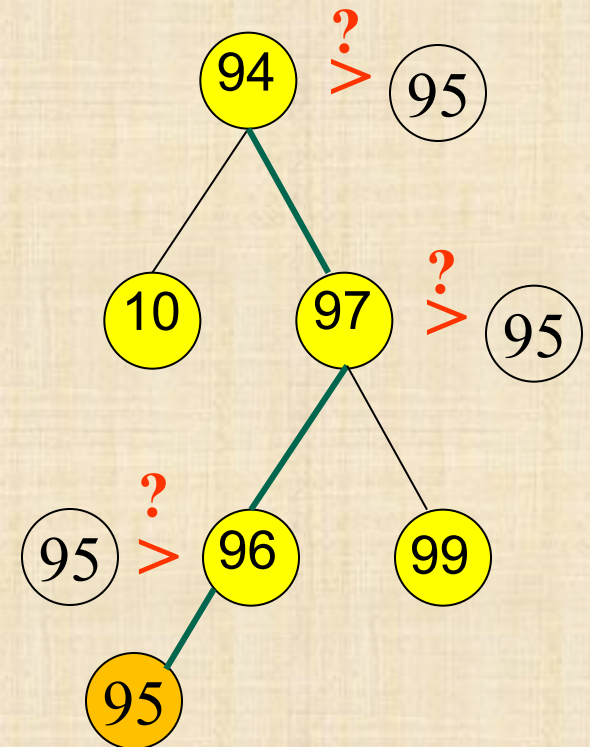
Example: Insert 95

Insert

Insert(T, x)

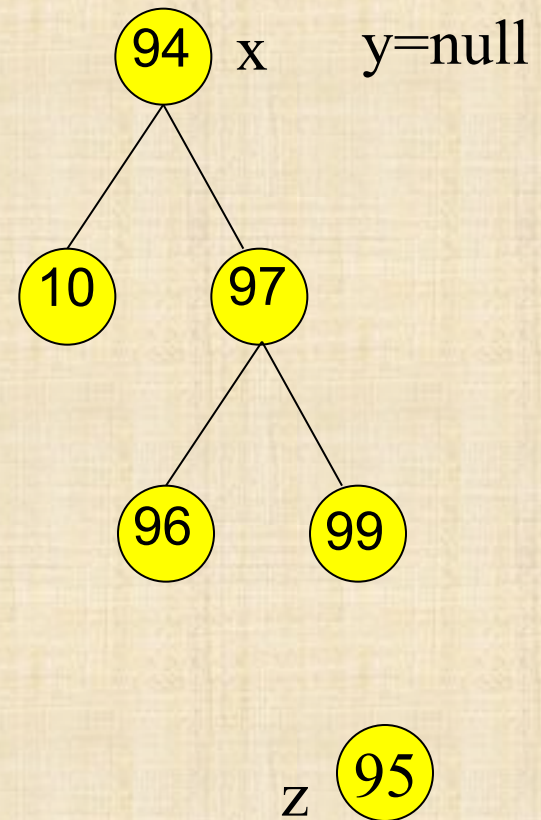
- ▷ Do a “Find” operation for x
- ▷ If x is found → announce key exists
- ▷ Else, “Find” stops at a *null* pointer
- ▷ Insert Node with x there

Example: Insert 95



Insert

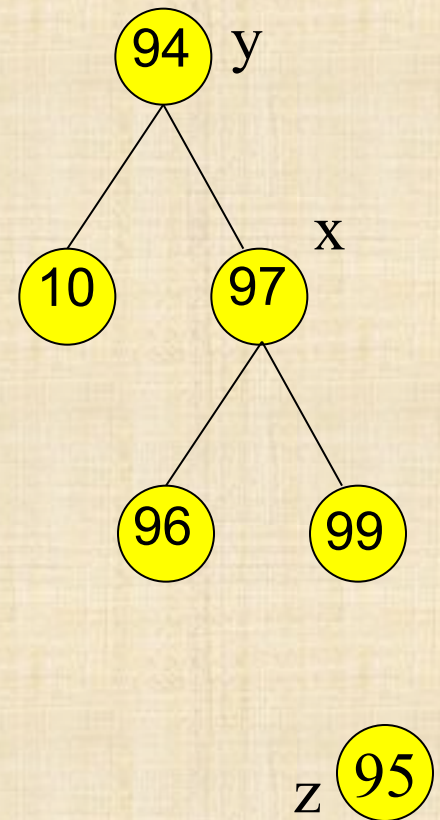
```
Tree-Insert(T, z)
y ← null      //prev node
x ← T         //current node
While x≠null
    y ← x
    if z.key < x.key
        then x ← x.left
    else x ← x.right
z.parent ← y
if y=null
    then T ← z
    elseif z.key < y.key
        then y.left ← z
    else y.right ← z
```



T: the root of a BST
z: a node with a key

Insert

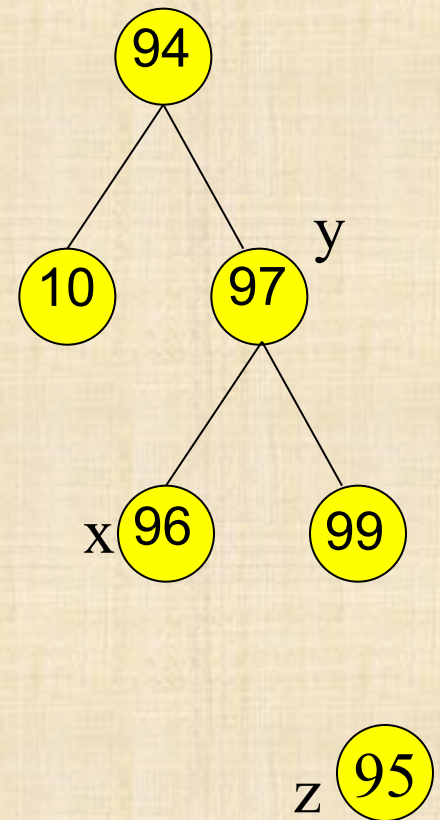
```
Tree-Insert(T, z)
y ← null           //prev node
x ← T              //current node
While x ≠ null
    y ← x
    if z.key < x.key
        then x ← x.left
    else x ← x.right
z.parent ← y
if y = null
    then T ← z
    elseif z.key < y.key
        then y.left ← z
    else y.right ← z
```



T: the root of a BST
z: a node with a key

Insert

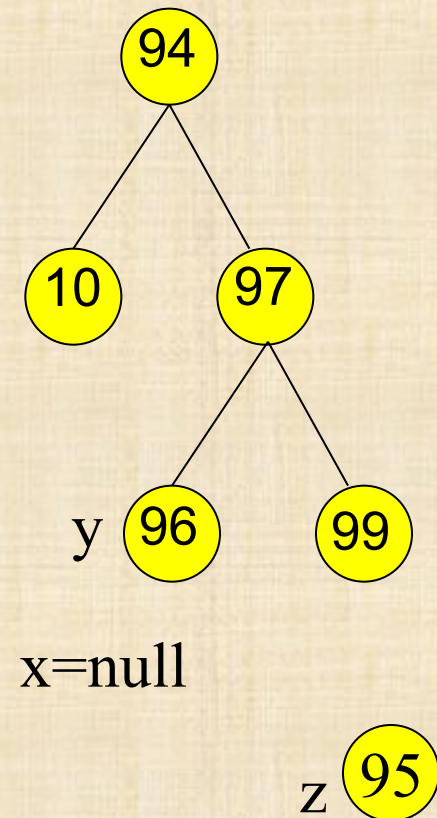
```
Tree-Insert(T, z)
y ← null           //prev node
x ← T              //current node
While x ≠ null
    y ← x
    if z.key < x.key
        then x ← x.left
    else x ← x.right
z.parent ← y
if y = null
    then T ← z
    elseif z.key < y.key
        then y.left ← z
    else y.right ← z
```



T: the root of a BST
z: a node with a key

Insert

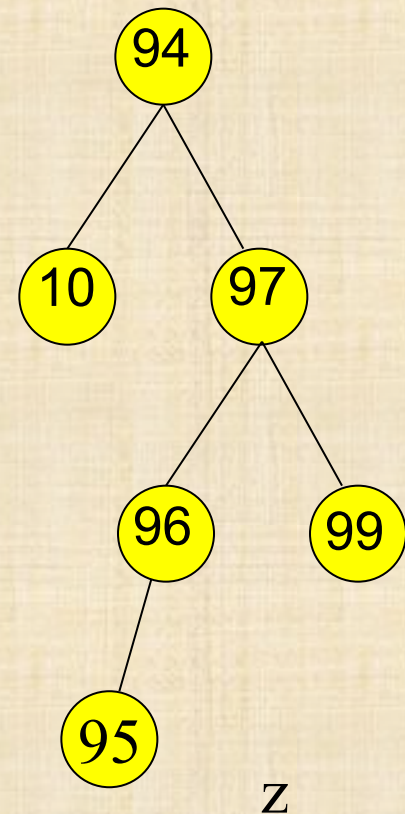
```
Tree-Insert(T, z)
y ← null           //prev node
x ← T              //current node
While x≠null
    y←x
    if z.key < x.key
        then x ← x.left
        else x ← x.right
z.parent ← y
if y=null
    then T ← z
    elseif z.key < y.key
        then y.left ← z
        else y.right← z
```



T: the root of a BST
z: a node with a key

Insert

```
Tree-Insert(T, z)
y ← null,
x ← T
While x ≠ null
    y ← x
    if z.key < x.key
        then x ← x.left
    else x ← x.right
z.parent ← y
if y = null
    then T ← z
    elseif z.key < y.key
        then y.left ← z
    else y.right ← z
```



Time complexity?

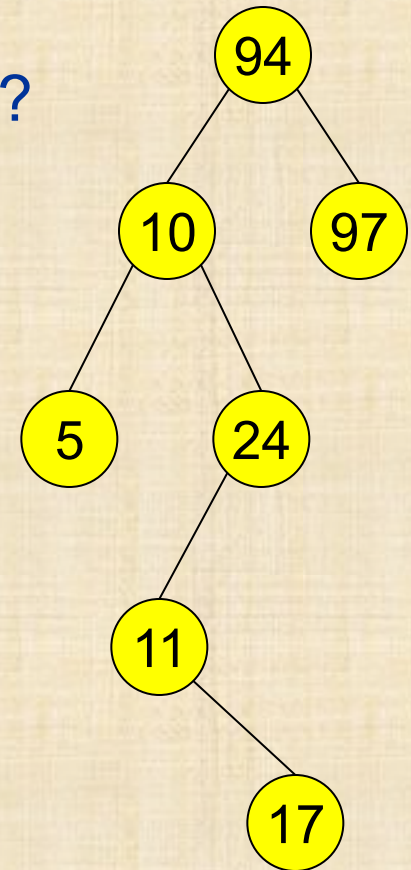
T: the root of a BST
z: a node with a key

Delete Operation

Problem: When deleting a node v ,
how should the 'hole' in the tree be filled?

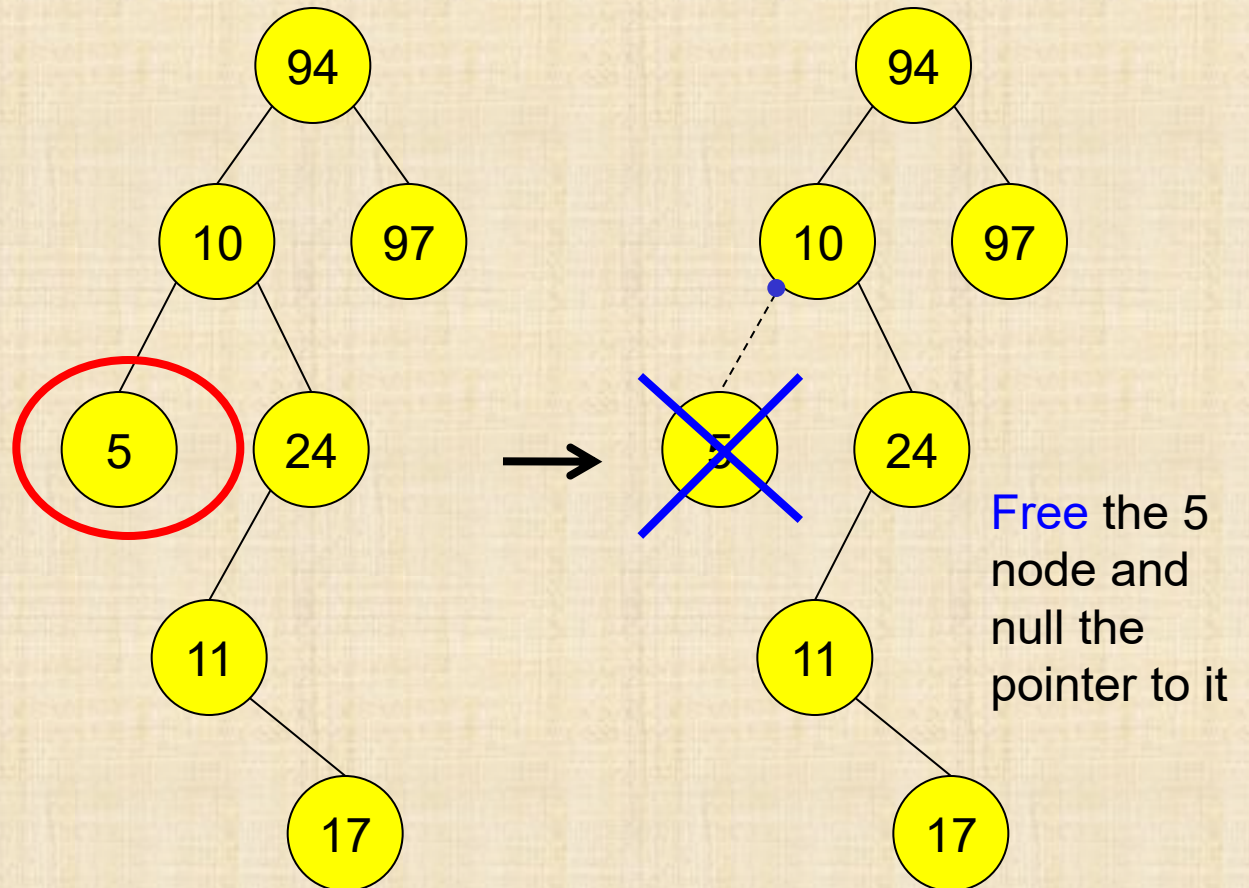
Solution:

- ▷ If v has no children, by *null*
- ▷ If v has **1** child, by that child
- ▷ If v has **2** children, by its successor
(must be in its right subtree)



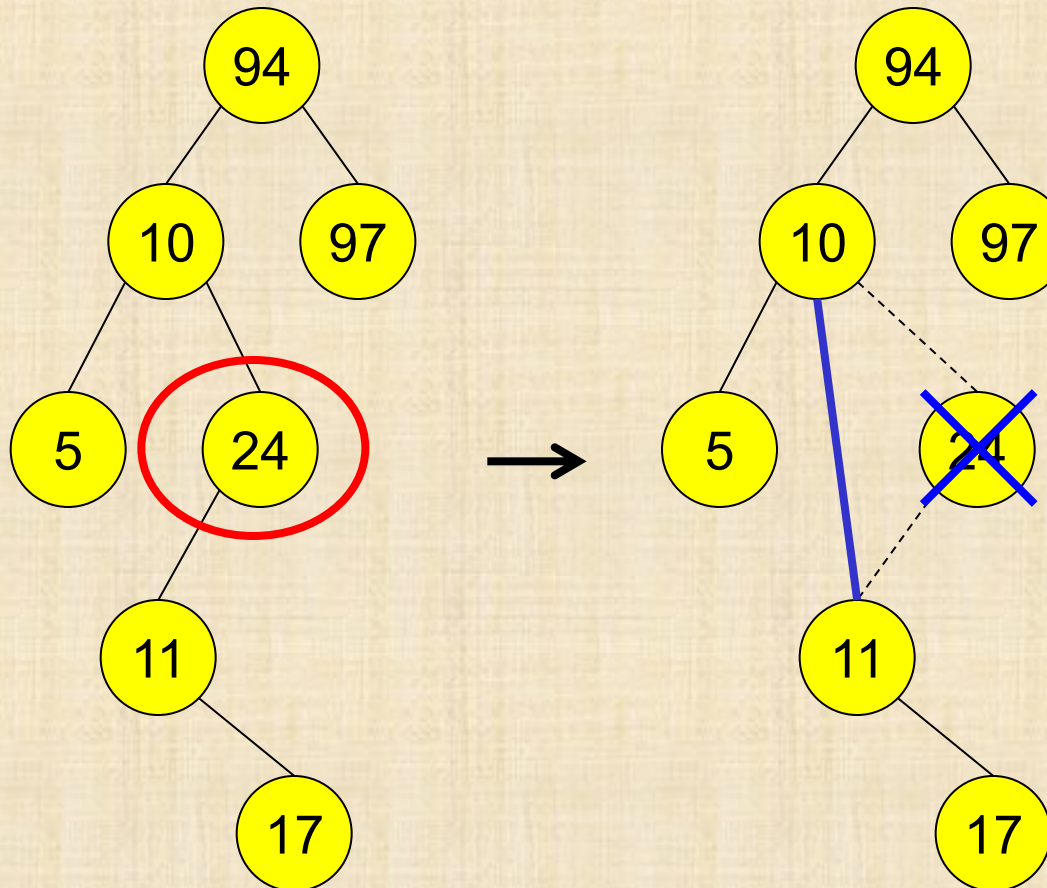
Time complexity?

Delete “5” - No children



▷ If v has no children, by null

Delete “24” - One child

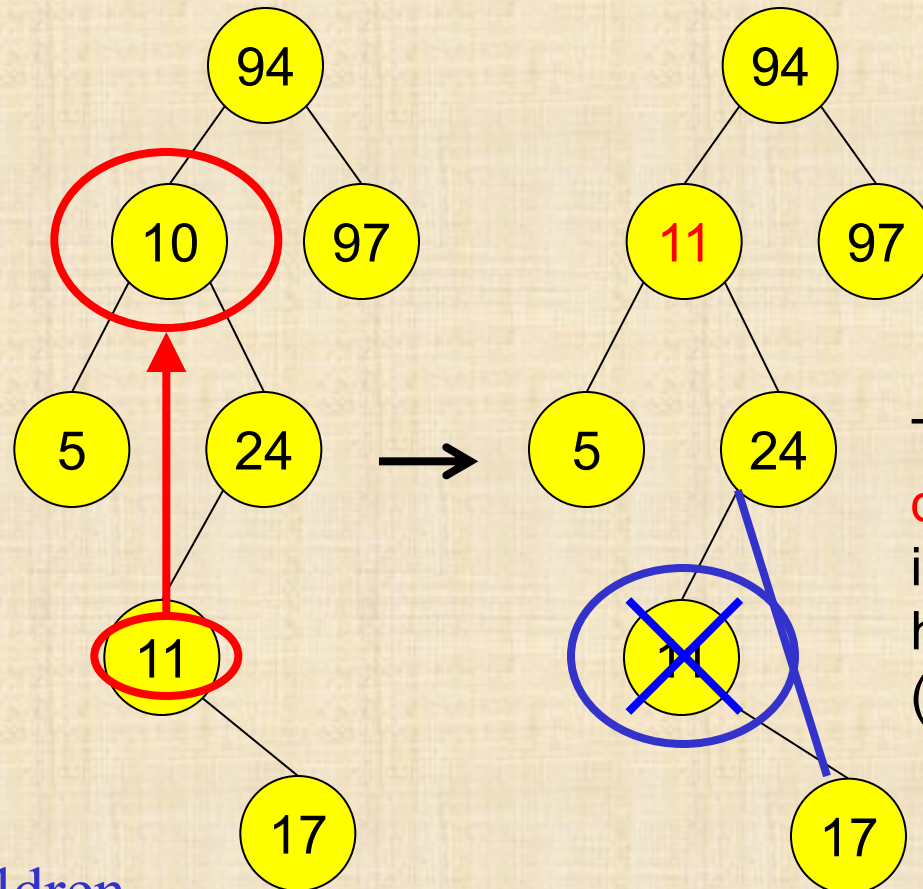


Free
the 24 node
and
replace the
pointer to it
with a
pointer to its
child

▷ If v has 1 child, by that child.

Delete “10” - two children

Copy the smallest value in right subtree into the node



Then (recursively) delete the node – it cannot have two children (why?)

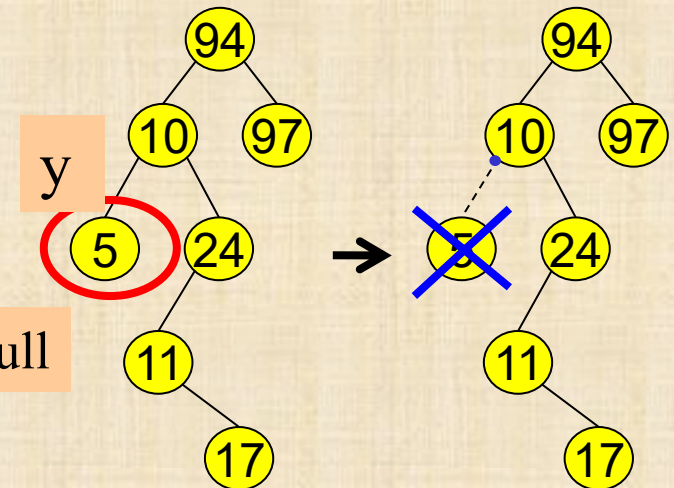
▷ If v has 2 children, by its successor

y – node we actually delete

x – single child of y (or null if no children)

z is the pointer to the node whose key and data we want to remove

```
Tree-Delete(T, z)
1  if z.left = null or z.right = null
2    then y ← z
3    else y ← Tree-Successor(z)
4  if y.left ≠ null
5    then x ← y.left
6    else x ← y.right
7  if x ≠ null
8    x.parent ← y.parent
9  if y.parent = null
10   then T.root ← x
11   else if y = (y.parent).left
12     then (y.parent).left ← x
13   else (y.parent).right ← x
14  if y ≠ z
15    then z.key ← y.key % also copy all data fields
16  return y
```



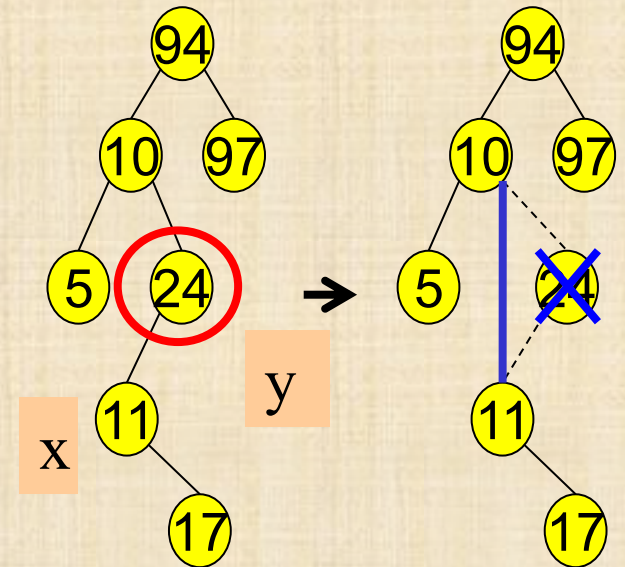
case 1: z is a leaf

y – node we actually delete

x – single child of y (or null if no children)

z is the pointer to the node whose key and data we want to remove

```
Tree-Delete(T, z)
1  if z.left = null or z.right = null
2  then y ← z
3  else y ← Tree-Successor(z)
4  if y.left ≠ null
5  then x ← y.left
6  else x ← y.right
7  if x ≠ null
8  x.parent ← y.parent
9  if y.parent = null
10 then T.root ← x
11 else if y = (y.parent).left
12 then (y.parent).left ← x
13 else (y.parent).right ← x
14 if y ≠ z
15 then z.key ← y.key % also copy all data fields
16 return y
```



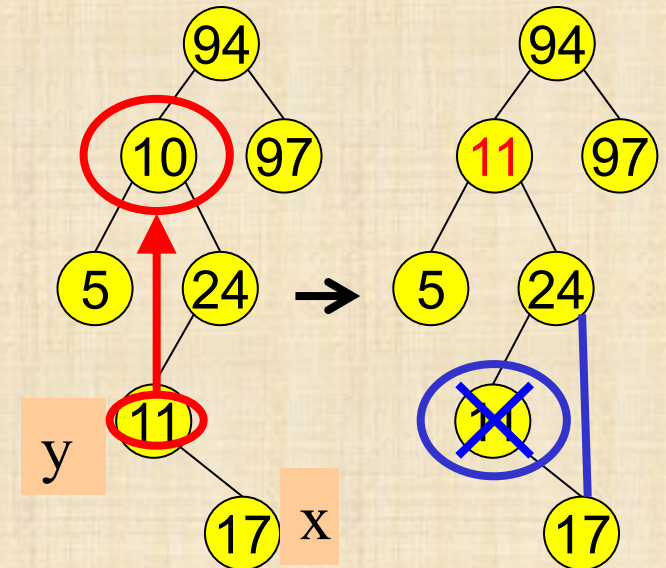
case 2: z has one child

y – node we actually delete

x – single child of y (or null if no children)

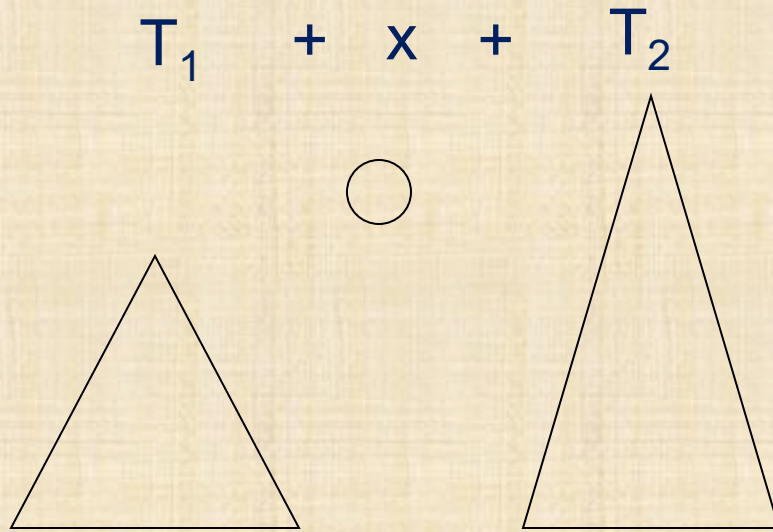
z is the pointer to the node whose key and data we want to remove

```
Tree-Delete(T, z)
1  if z.left = null or z.right = null
2    then y ← z
3    else y ← Tree-Successor(z)
4  if y.left ≠ null
5    then x ← y.left
6    else x ← y.right
7  if x ≠ null
8    x.parent ← y.parent
9  if y.parent = null
10   then T.root ← x
11   else if y = (y.parent).left
12     then (y.parent).left ← x
13   else (y.parent).right ← x
14  if y ≠ z
15    then z.key ← y.key % also copy all data fields
16  return y
```



case 3: z has two children

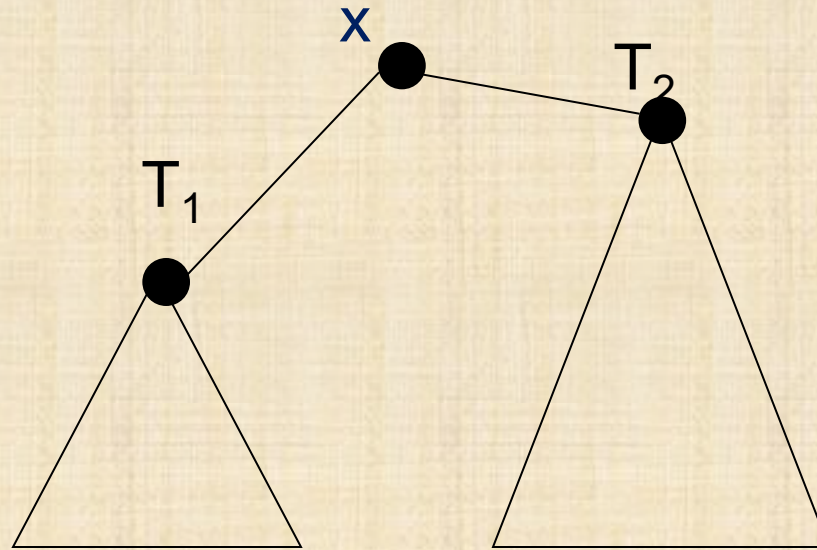
Join(T_1, x, T_2)



Input: Two binary search trees, T_1 , T_2 , and a node x .

It is known that keys in $T_1 < \text{key of } x < \text{keys in } T_2$

Join(T_1, x, T_2)



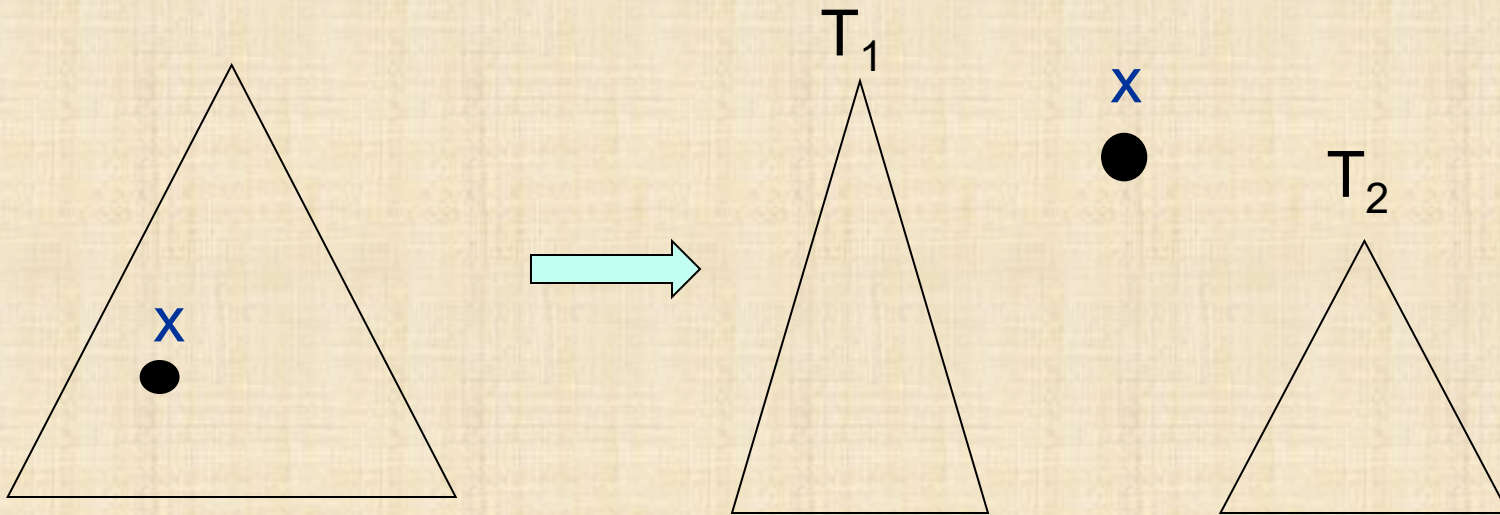
Output: One BST consisting of $T_1 \cup \{x\} \cup T_2$.

Solution: make x the root

T_1 is the left subtree, T_2 is the right subtree

Complexity: $O(1)$

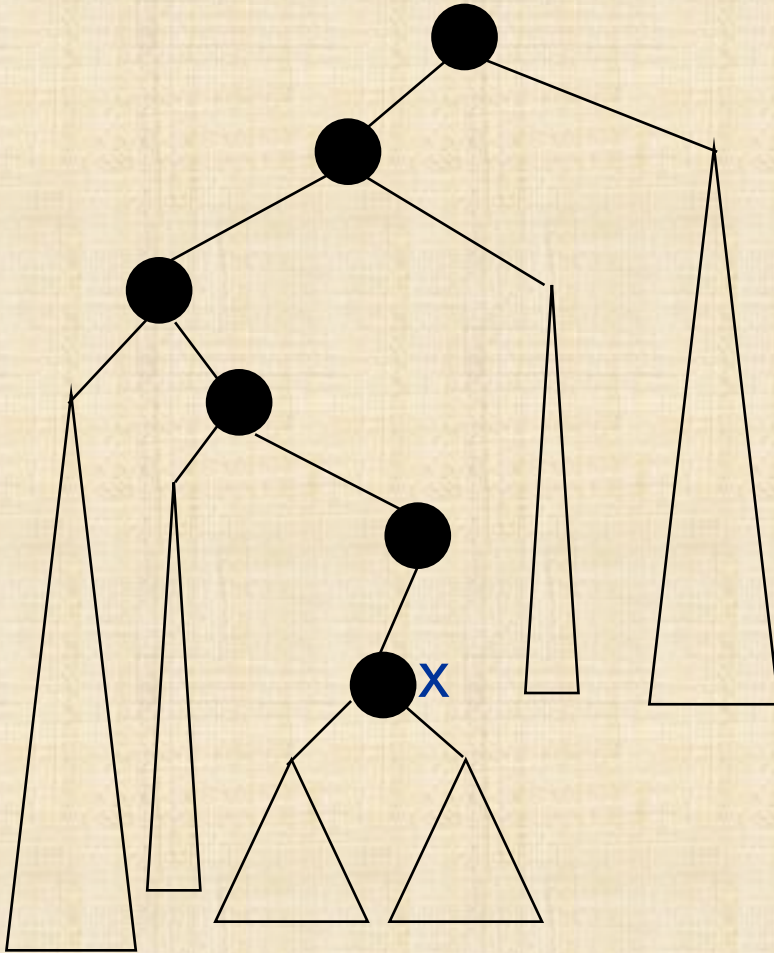
Split(T, x)



Input: A BST, one element in the tree whose key is x .

Output: Split T into two BSTs and the node whose key is x . keys in $T_1 < \text{key of } x < \text{keys in } T_2$

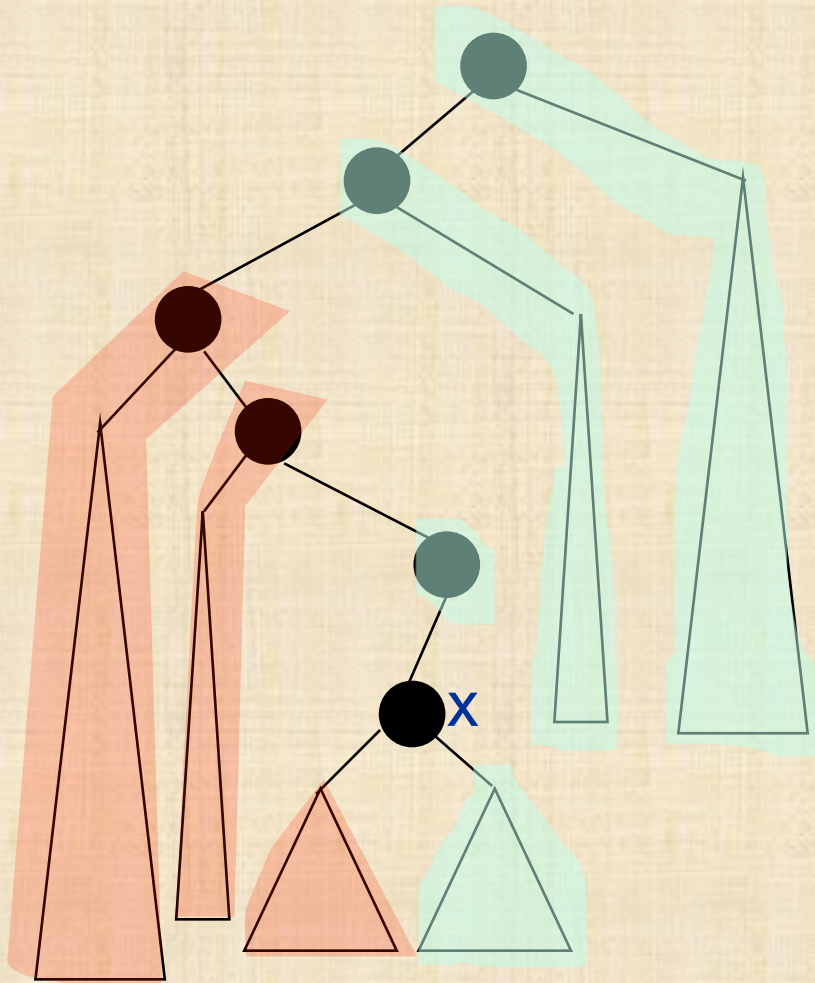
Split(T, x) into $T_1 + x + T_2$



Where in T are the elements smaller than x ?

Where in T are the elements larger than x ?

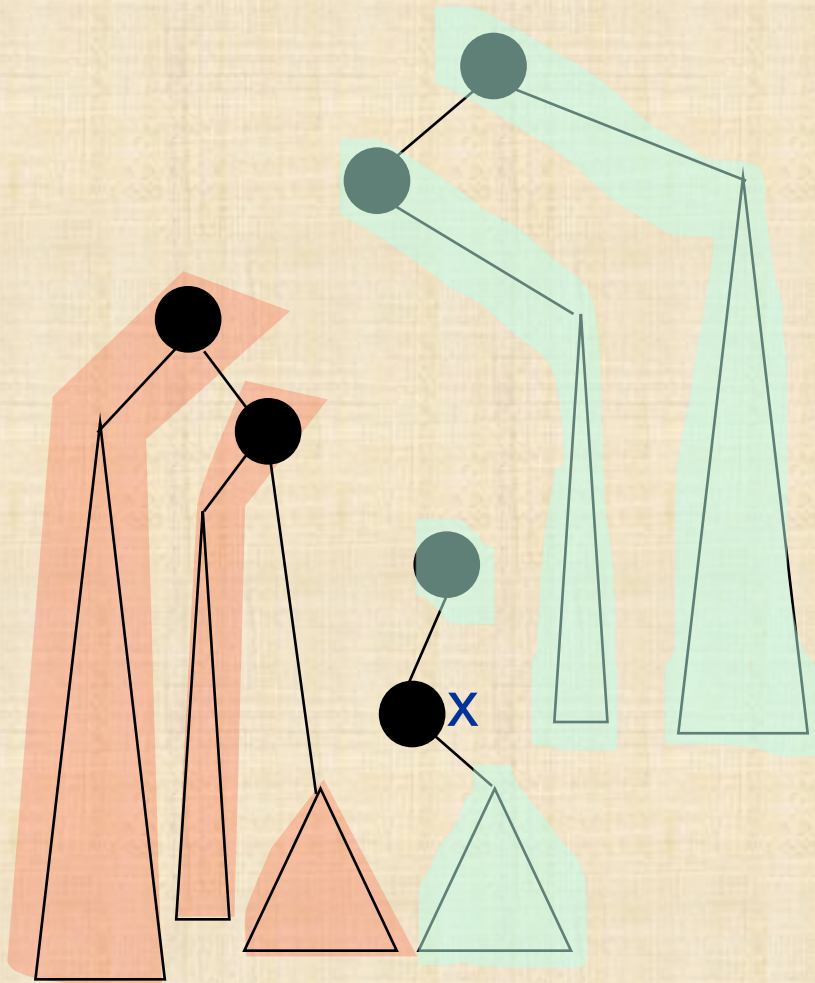
Split(T, x) into $T_1 + x + T_2$



Where in T are the elements smaller than x ?

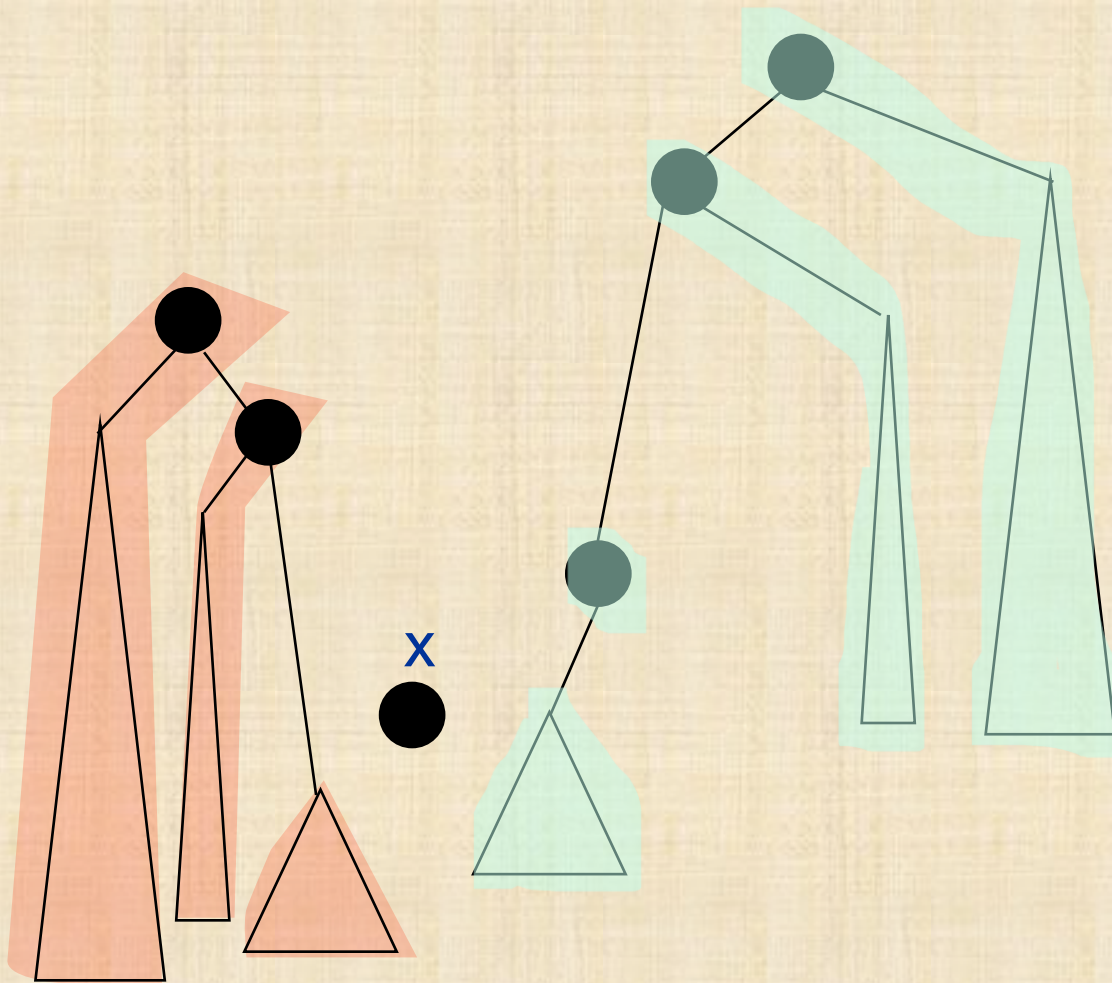
Where in T are the elements larger than x ?

Split(T, x) into $T_1 + x + T_2$



Join the left subtree of x with all trees to the left of the path from the root to x to form the left tree T_1 .

Split(T, x) into $T_1 + x + T_2$



Join the left subtree of x with all trees to the left of the path from the root to x to form the left tree T_1 .

Join the right subtree of x with all trees to the right of the path from the root to x to form the right tree T_2

Complexity: $O(h)$

Time Complexity

- ▷ All operations we considered so far have $O(h)$ time complexity where h is height of the tree
- ▷ Worst case: h might be $\Omega(n)$
 - ▷ When?
- ▷ *Best case: $O(\log n)$*
 - ▷ When?

Balanced Trees – we'll discuss later

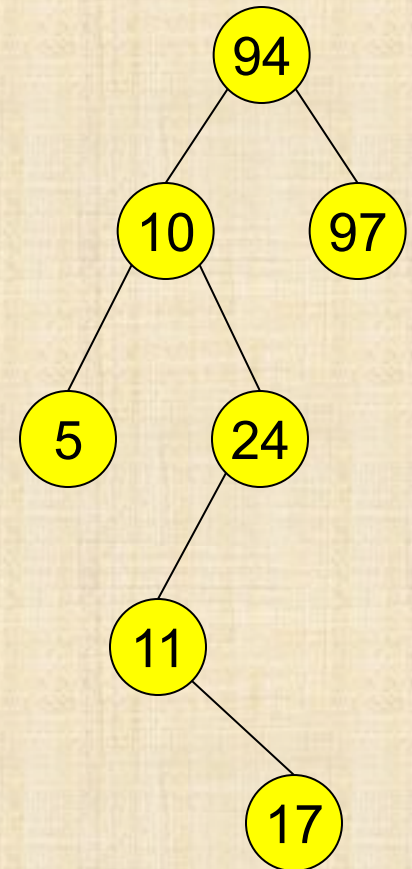
Traversing General Binary Trees

- ▷ Three useful traversals (recursive definitions):
 1. **Preorder (DLR)**: Node, then Children recursively (starting with the left child)
 2. **Inorder (LDR)**: Left child recursively, Node, Right child recursively
 3. **Postorder (LRD)**: Children recursively, then Node
- ▷ Traversals can be extended to general ordered (non-binary) trees

Inorder tree traversal -

```
inorder-tree-walk(T)  
if T≠null then  
    inorder-tree-walk(T.left)  
    print T.key  
    inorder-tree-walk(T.right)
```

(LDR)



Inorder tree traversal - example

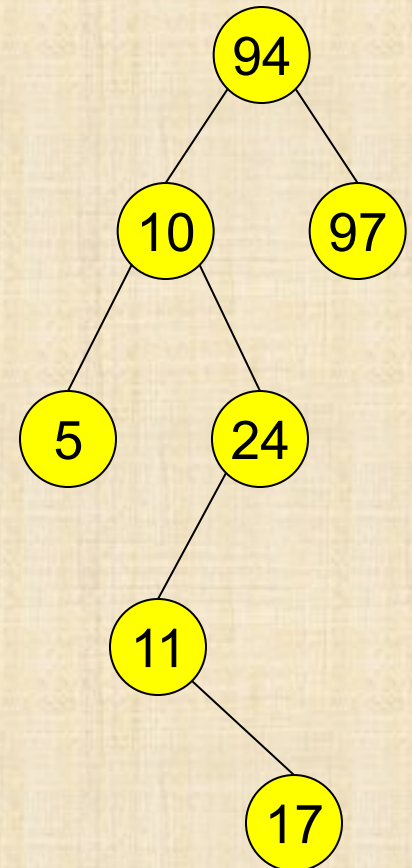
(LDR)

▷ Output: 5, 10, 11, 17, 24, 94, 97

What is the output on a binary search tree?

Time complexity?

What can we say about the time complexity of building a BST?



Postorder tree traversal

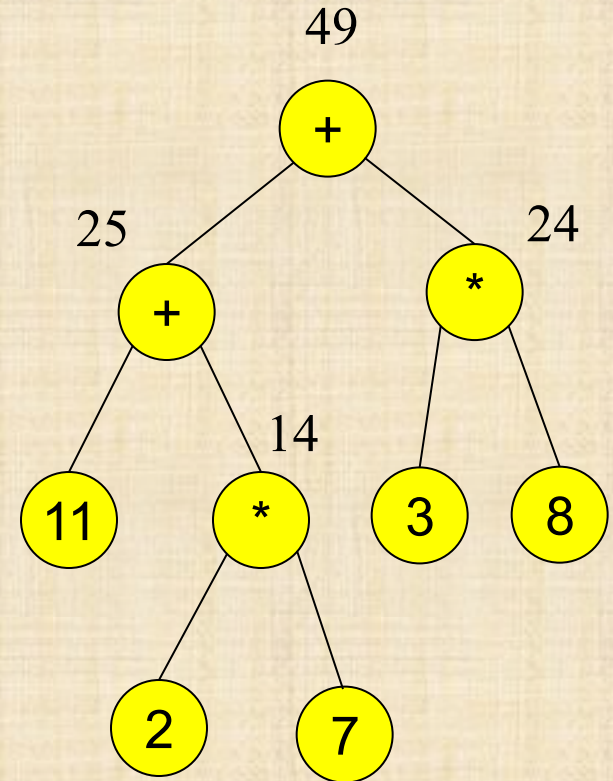
```
postorder-tree-walk(T) (LRD)  
if T≠null then  
    postorder-tree-walk(T.left)  
    postorder-tree-walk(T.right)  
    print T.key
```

Evaluating a mathematical expression by postorder

Consider a full binary tree in which every internal node represents a binary operation, and every leaf represents a number.

Every subtree has a value:

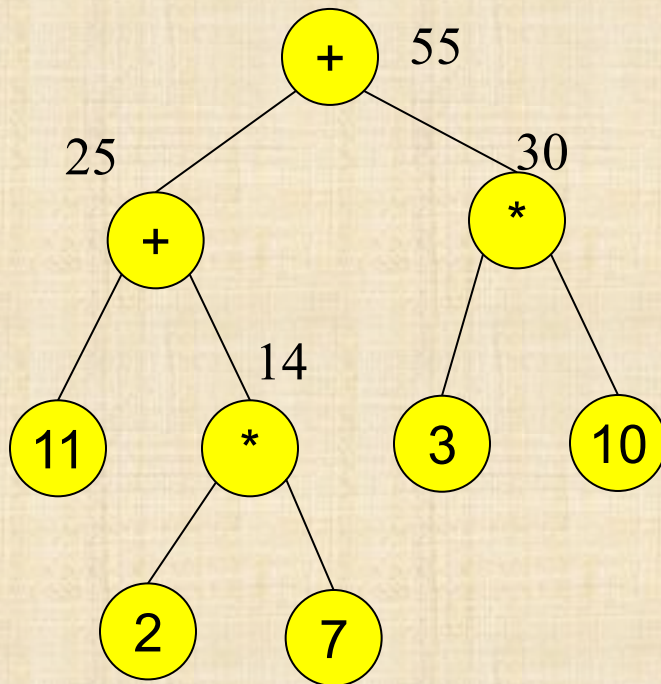
- The value of a leaf is the number it holds
- The value of a subtree rooted at an internal node v is the result of the operation at v performed on the values of the subtrees of v



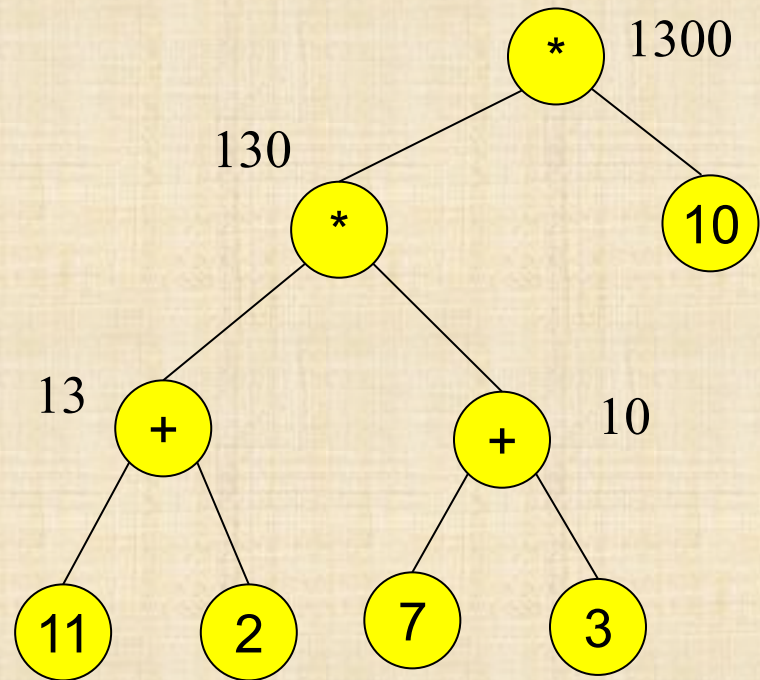
What is the meaning of a postorder traversal?

Evaluating a mathematical expression by postorder

$$(11+(2*7))+(3*10)$$

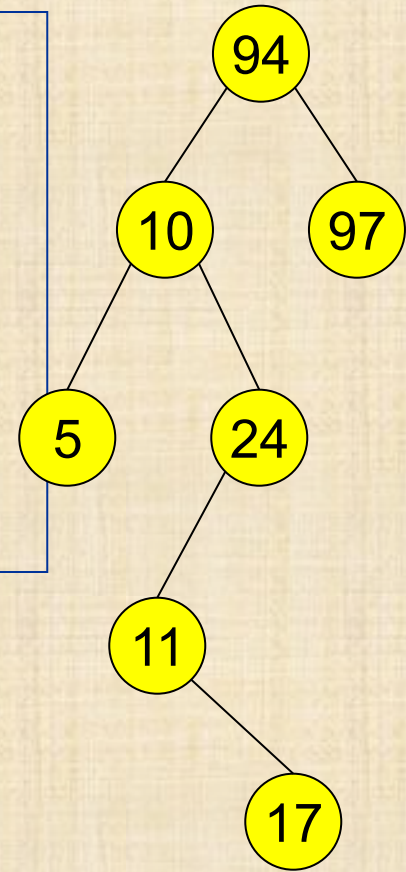


$$((11+2)*(7+3))*10$$



Preorder tree traversal

```
preorder-tree-walk(T) (DLR)  
if T≠null then  
    print T.key  
    preorder-tree-walk(T.left)  
    preorder-tree-walk(T.right)
```



Output: 94,10,5,24,11,17,97

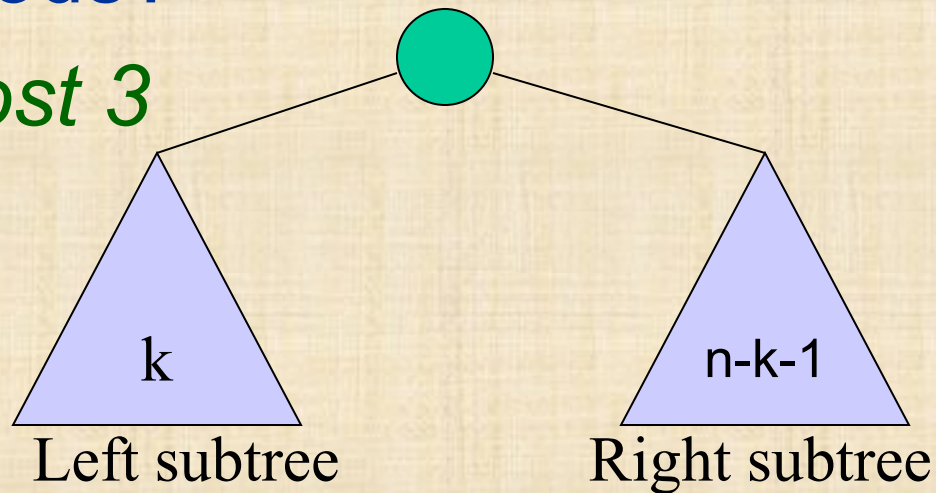
Applications for preorder

- ▷ Creating a copy of the tree
- ▷ Serializing a binary search tree –
 - ▷ When you want to ‘save’ a tree structure to file
- ▷ Directory structure printing
 - ▷ Because you want to print the root directory before you print its contained files/directories

Traversing Binary Trees

- ▷ How many times is each node visited by any of the traversal methods?

at most 3



Building a BST

- ▷ Assume we have n objects that need to be stored in a binary search tree
- ▷ How can we build the tree?

Building a BST

- ▷ Naïve insert:
 - ▷ $\Theta(nh)$ where h is the tree height
 - ▷ Best case: $\Theta(n \log n)$
 - ▷ Worst case $\Theta(n^2)$
- ▷ Can we do better than $\Theta(n \log n)$?
 - ▷ Unsorted keys - comparison based: NO!
Contradicts the $\Omega(n \log n)$ sorting lower bound
 - ▷ Sorted keys: Yes

Unsorted Keys - Comparison Based

- ▷ **Claim:** inserting n unsorted keys into a BST, using only comparison takes $\Omega(n \log n)$ time
 - ▷ That is, it is impossible to build a BST in $o(n \log n)$

▷ **Proof (by contradiction):**

- ▷ Assume it is possible to build a BST in $o(n \log n)$ (strictly less than $n \log n$)
- ▷ Perform inorder in time $\Theta(n)$ to obtain the n sorted keys
- ▷ It follows that we can sort n keys in $o(n \log n)$ using a comparison-based algorithm
- ▷ Contradiction!
(to the lower bound of comparison-based sorting algorithm)

Build a BST: Sorted Keys

- ▷ Take the median to be the root
- ▷ Repeat recursively
- ▷ Every node is added in $\Theta(1)$ time
- ▷ Worst-case is $\Theta(n)$

Build a balanced BST from a sorted array

BuildBST(A, q, r)

if $q > r$ then return null

else

$$m = \left\lfloor \frac{q+r}{2} \right\rfloor$$

create a new node v

v.data=A[m]

v.left=BuildBST(A, q, m-1)

v.right=BuildBST(A, m+1, r)

return v

A: Input array
q,r: range of elements
that should be included
in BST

Summary

- ▷ Binary Search Trees
 - ▷ Find, Insert, Delete, Min, Max, Successor, Predecessor, Join, Split operations
 - ▷ Tree traversals
 - ▷ Next up: **Augmented data** – supporting additional operations
 - ▷ Important and still open (until next week): how can we keep the tree balanced?

Additional Operations

- ▷ Rank - What is the *rank* of a given key?
- ▷ Select - What is the key with a *rank* k ?
- ▷ Compute the *sum* of values of all elements that are larger than a given key?
- ▷ What is the average of the 10% lower keys?
- ▷

Selection

Return the element with *rank* k

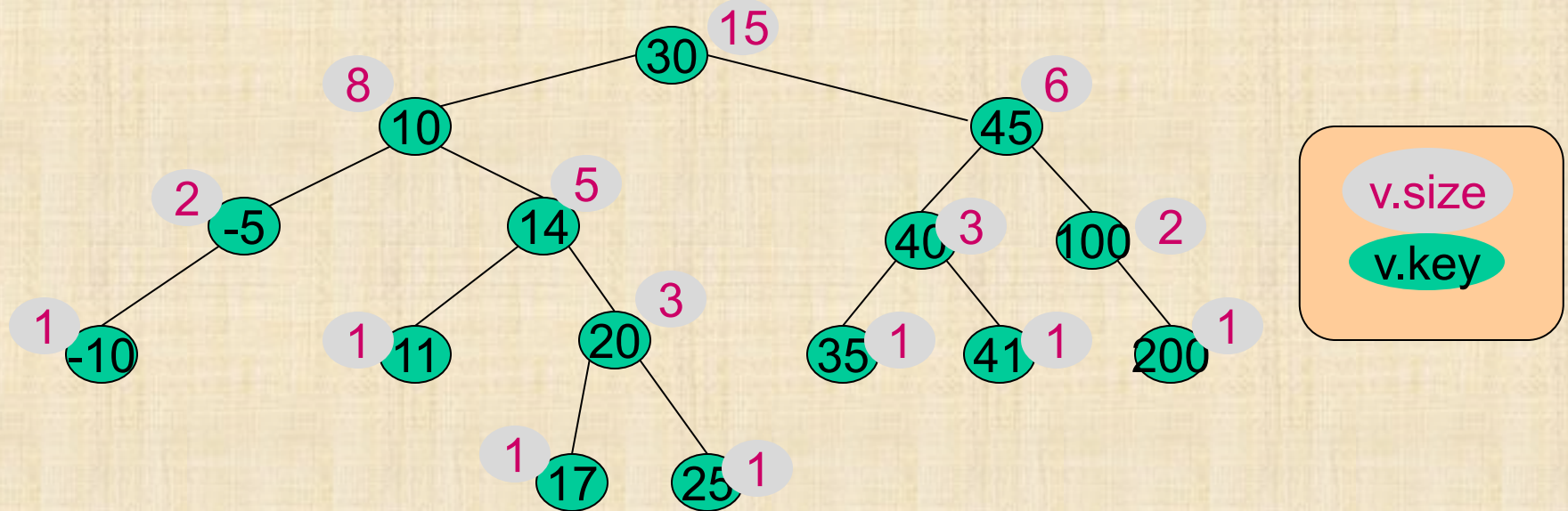
- ▷ For a sorted array A : $A[k]$
- ▷ For an unsorted array:
 - ▷ Quick selection
 - ▷ Median-of-medians (MMSelect)
- ▷ What about a BST?
 - ▷ Naïve solution: worst case $\Theta(n)$
 - ▷ We will see a more efficient solution using **augmented data**

Augmented Data

- ▷ Additional data in each node can be used to efficiently answer different questions
- ▷ Requires updating this data whenever the tree is updated (*insert, delete, ...*).
 - ▷ These updates should have the same complexity as insert / delete in a BST ($\Theta(h)$)

Size augmentation

- ▷ Add one more entry per node
- ▷ $v.size$: the number of nodes in the subtree rooted at v
- ▷ Each node v stores: the key, the data, and $v.size$

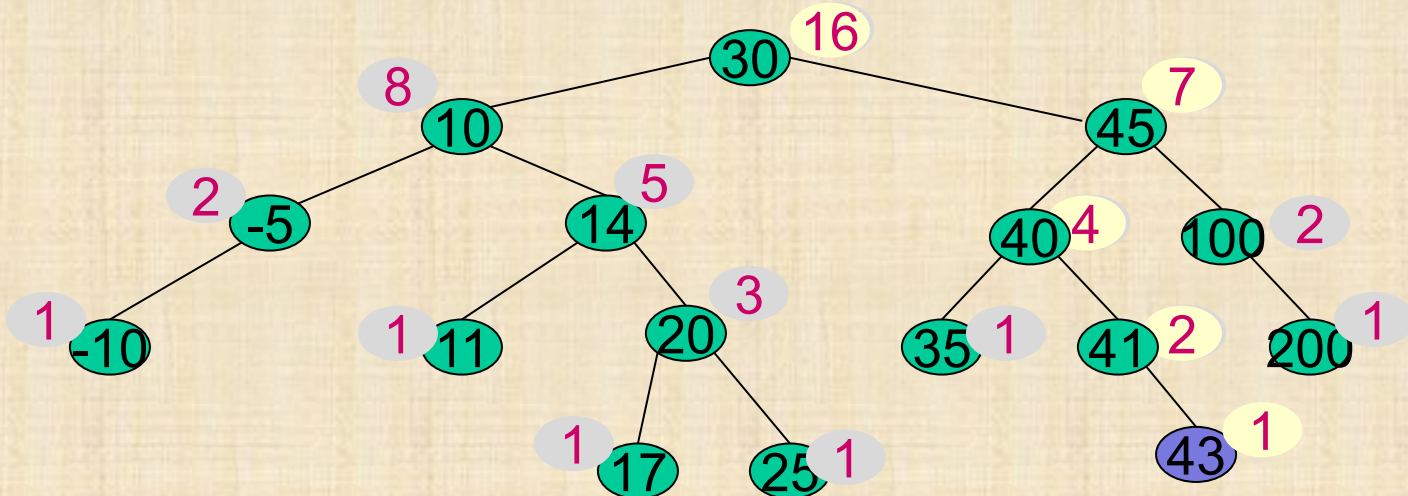
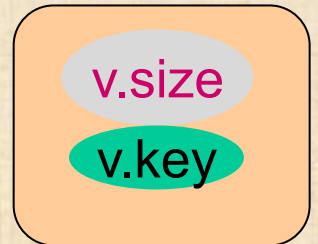
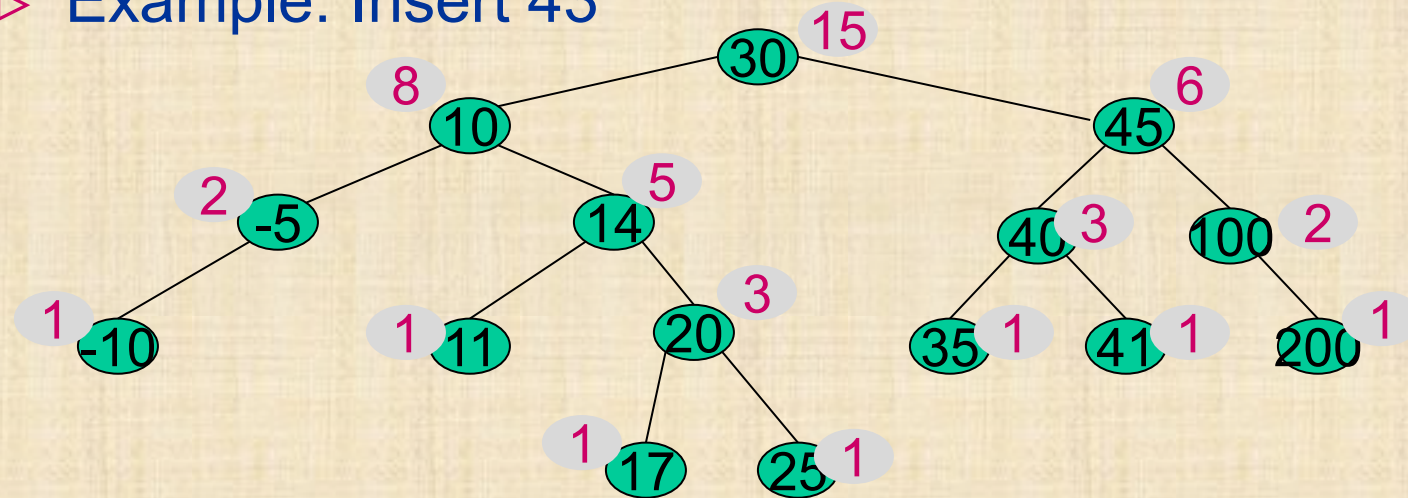


Updating Size Field

- ▷ Node insert:
 - ▷ A new node, x , is always a leaf: $x.size=1$
 - ▷ Increase by 1 all $v.size$ along the path from x to the root
- ▷ Node delete:
 - ▷ Decrease by 1 all $v.size$ along the path from the root to the node
 - ▷ Note: in case of 2 children – the deleted node is the successor

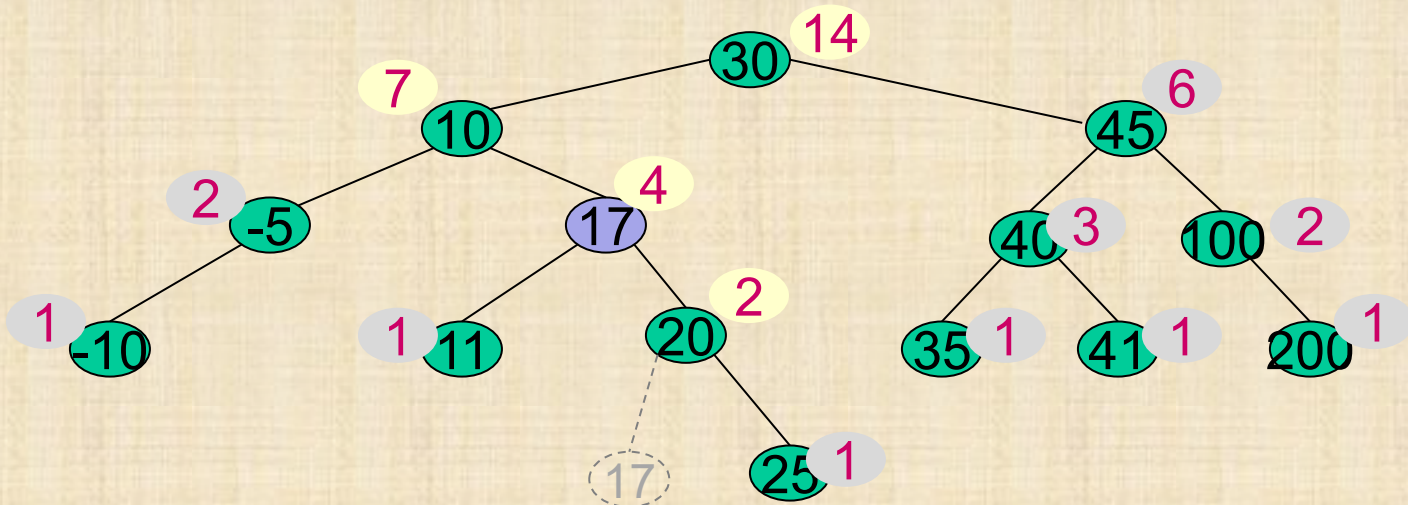
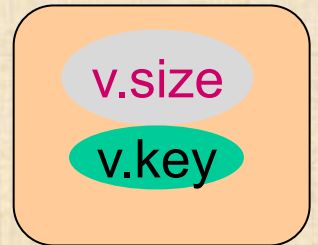
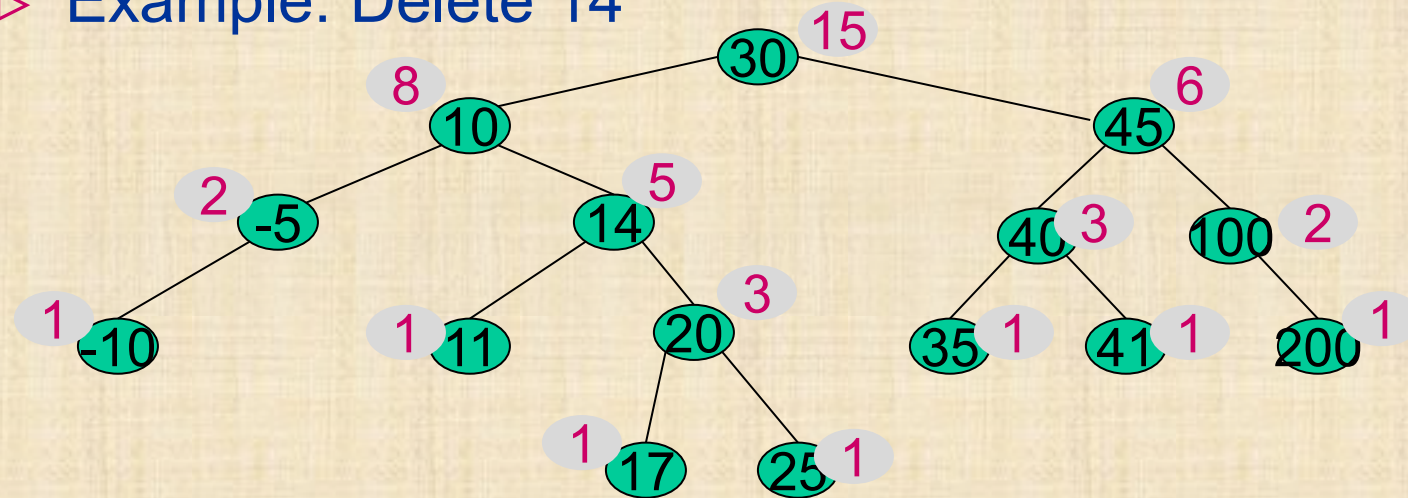
Updating Size Field

▷ Example: Insert 43



Updating Size Field

▷ Example: Delete 14



SELECT(T, k)

Goal: Return a pointer to the node containing the element with rank k in T

Idea:

▷ $r[x]$ - the rank of node x in the subtree rooted at x

▷ Use the size field:

$$r[root] = (root.left).size + 1$$

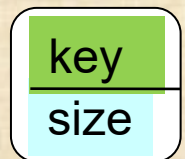
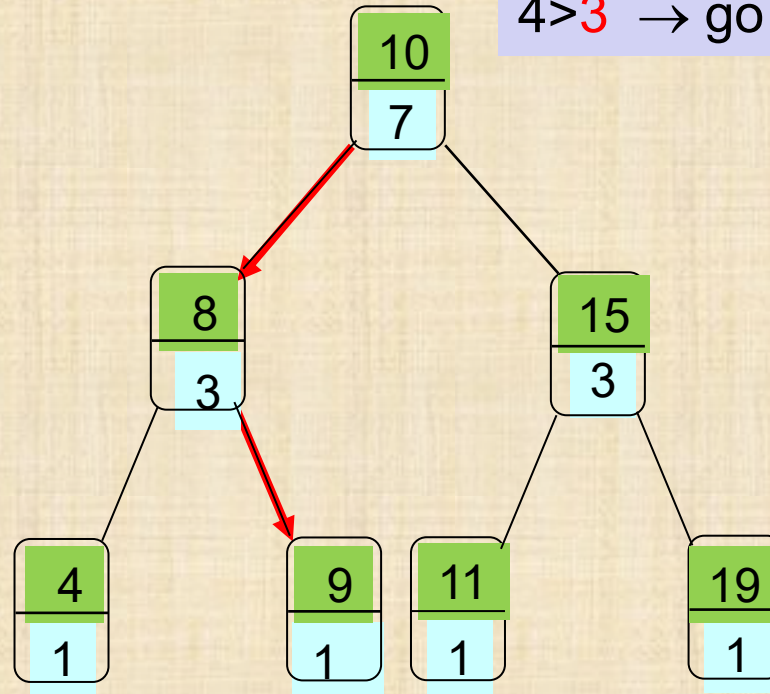
Example: $\text{SELECT}(T, k)$

Abused notations:
We use the 'key' as a
pointer to the node

$\text{SELECT}('10', 3)$

$\text{SELECT}('10', 3)$
 $r[10] = 3 + 1 = 4$
 $4 > 3 \rightarrow \text{go left}$

$\text{SELECT}('8', 3)$
 $r[8] = 1 + 1 = 2$
 $2 < 3 \rightarrow \text{go right}$



$\text{SELECT}('9', 1)$
 $r[9] = 0 + 1 = 1$.
Found! $\rightarrow 1 = 3 - 2$

SELECT(T, k)

Initial call: SELECT(T, k)

SELECT(x, k)

if ($x = \text{null}$), return null

if $x.\text{left} \neq \text{null}$

 then $r \leftarrow (x.\text{left}).\text{size} + 1$

else $r \leftarrow 1$

if $k = r$

 then return x

else if $k < r$

 then return *SELECT*($x.\text{left}, k$)

else return *SELECT*($x.\text{right}, k - r$)

x : a node

k : rank in a subtree
with x as a root

% rank of x in the
subtree rooted at x

Time complexity of Select: $O(h)$

RANK(T, x)

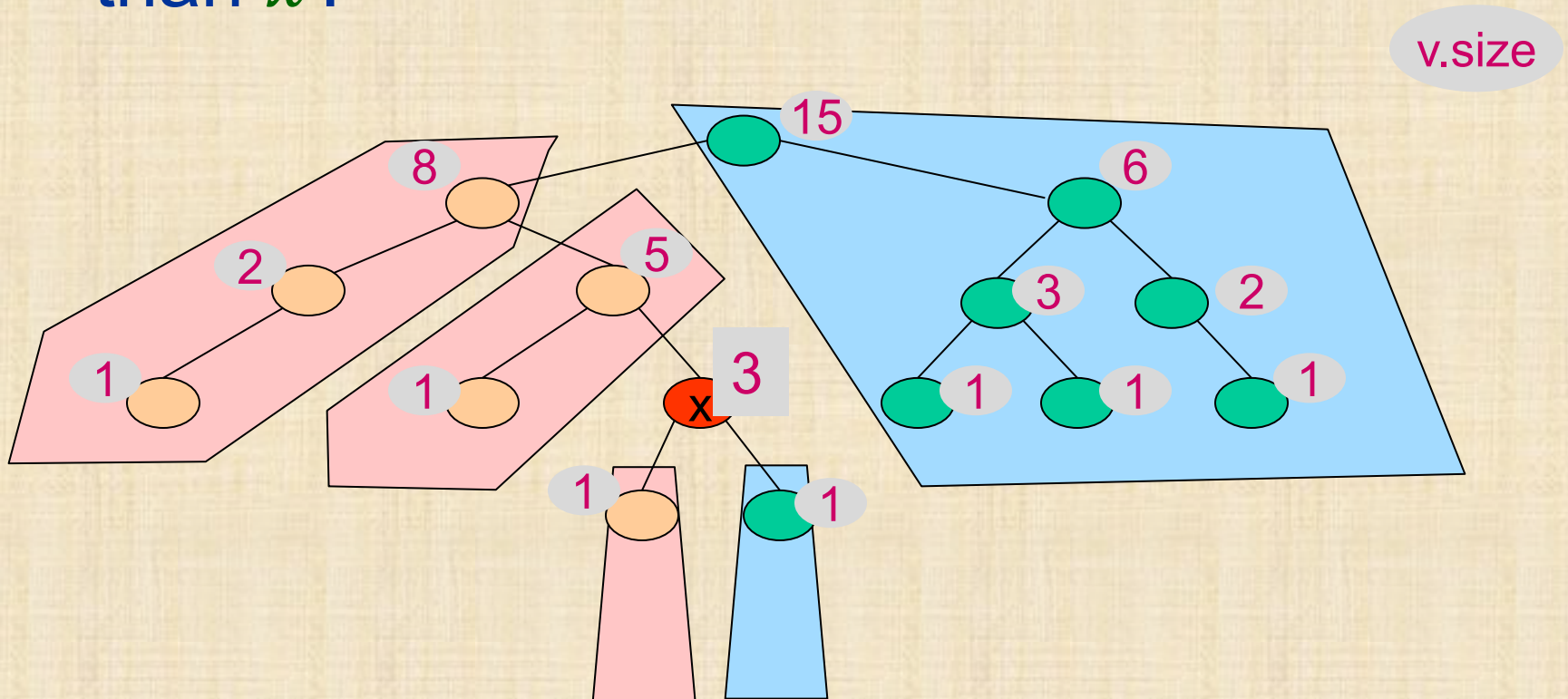
Goal: given a pointer x to a node in a BST T return the rank of x in T

Idea:

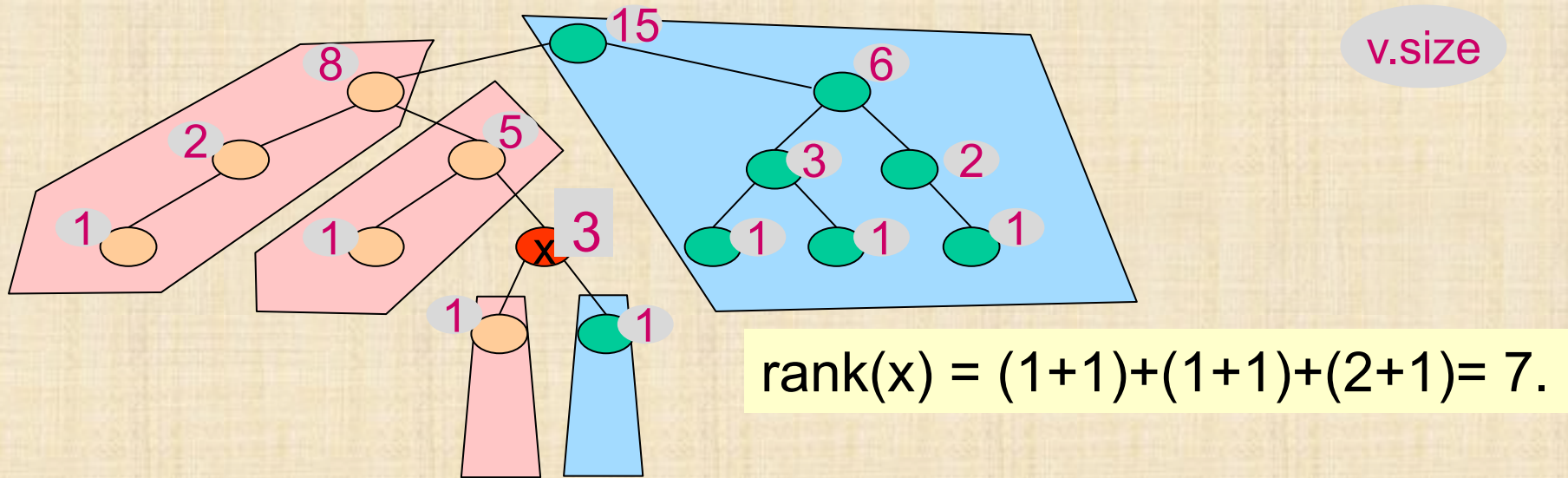
- ▷ Use size augmentation
- ▷ Count the number of keys in T that are smaller than the key of x :
 - ▷ The size of the left subtree of x
 - ▷ What else?

$\text{RANK}(T, x)$

- ▷ Where in the tree are the values smaller than x ?



Rank(T, x)



1. Add the elements in the left subtree of x , $+1$ for x itself
2. Consider the path p from x to the root:

For every v in p (including x): If v is a right child, then add the size of v 's parent's left subtree $+ 1$ for the parent

Rank(T, x)

T : root of the tree
 x : a node

RANK(T, x)

If $x.\text{left} \neq \text{null}$ % r : the rank of x in the tree rooted in x

 then $r \leftarrow (x.\text{left}).\text{size} + 1$

 else $r \leftarrow 1$

$y \leftarrow x$

while $y.\text{parent} \neq \text{null}$ do % y a pointer that traverse up the tree

 if $y = (y.\text{parent}).\text{right}$

 then if $(y.\text{parent}).\text{left} \neq \text{null}$

 then $r \leftarrow r + ((y.\text{parent}).\text{left}).\text{size} + 1$ % rank of y in the

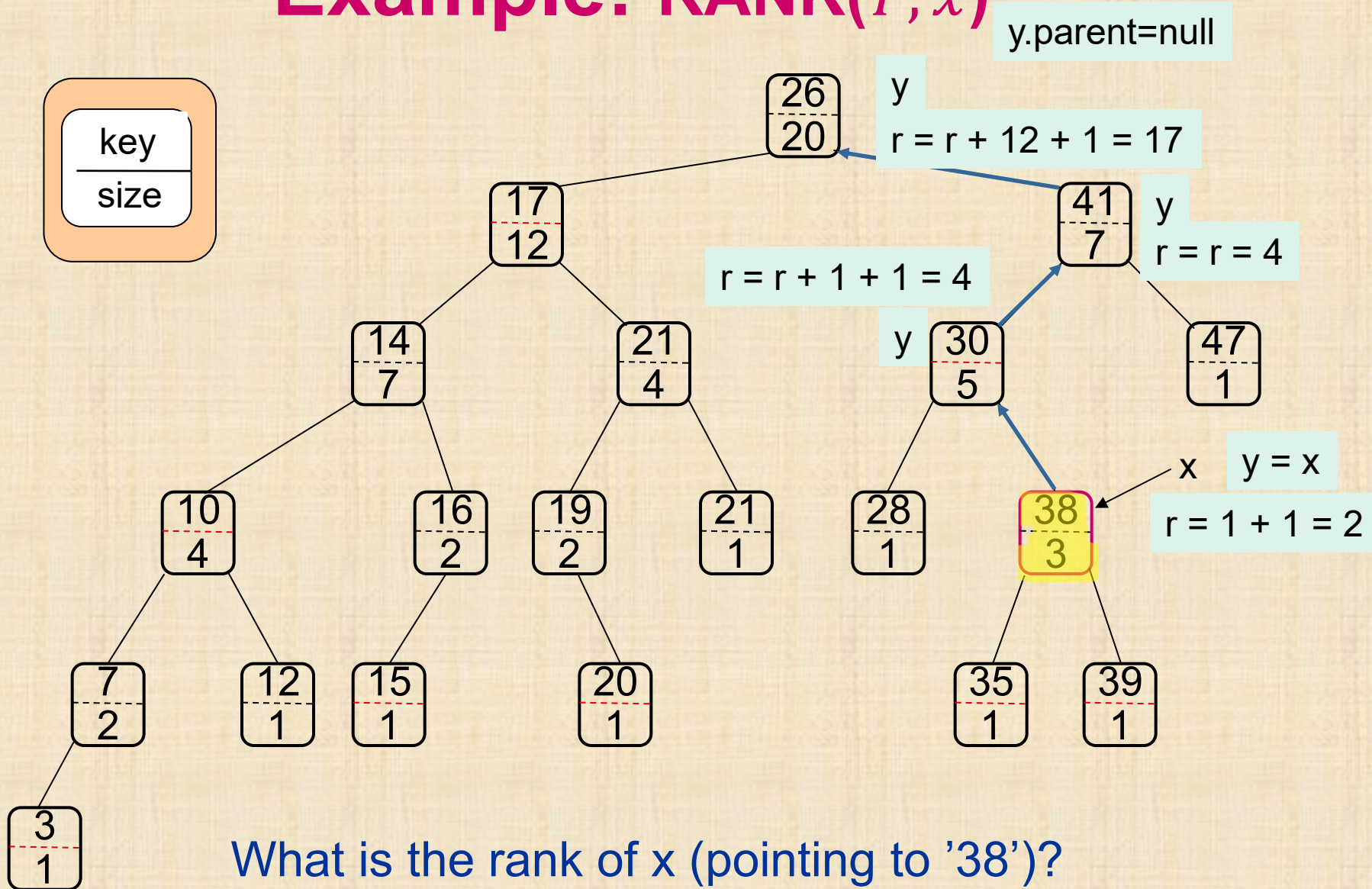
 else $r \leftarrow r + 1$ tree rooted at y

$y \leftarrow y.\text{parent}$

return r

Time complexity?

Example: $\text{RANK}(T, x)$



What is the rank of x (pointing to '38')?

Sum of Keys $< x$

Goal: given a value x return the sum of keys smaller than x in the tree

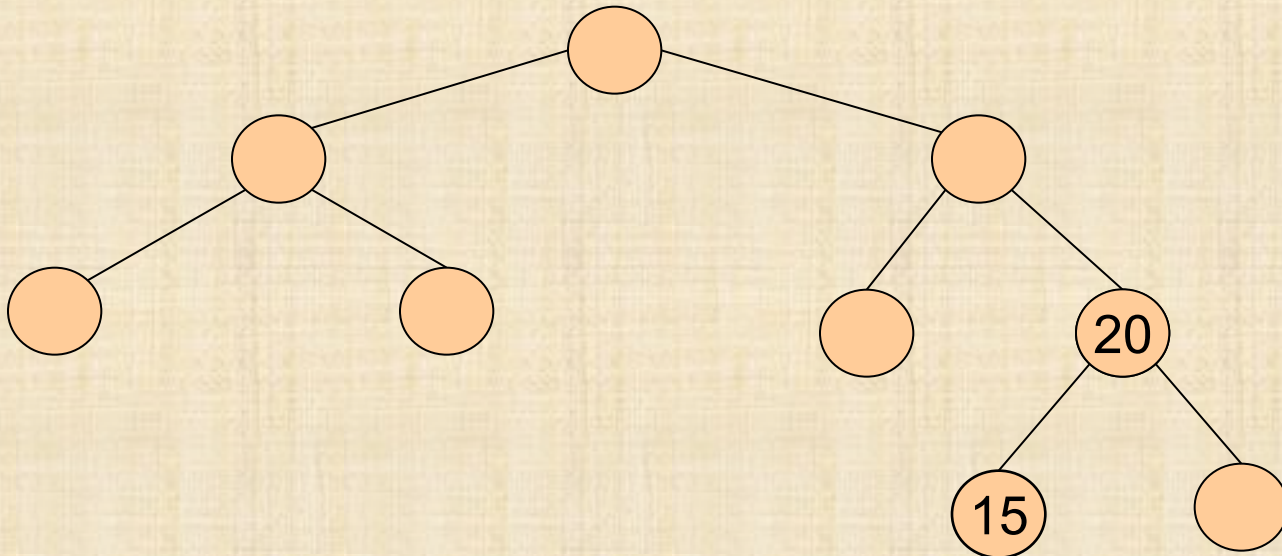
Idea:

- ▶ Use augmented BST data structure:
 - ▶ Store in v a field $v.sum$ = the sum of all keys in the subtree rooted at v
- ▶ Use it to answer sum-related queries

Sum of Keys $< x$

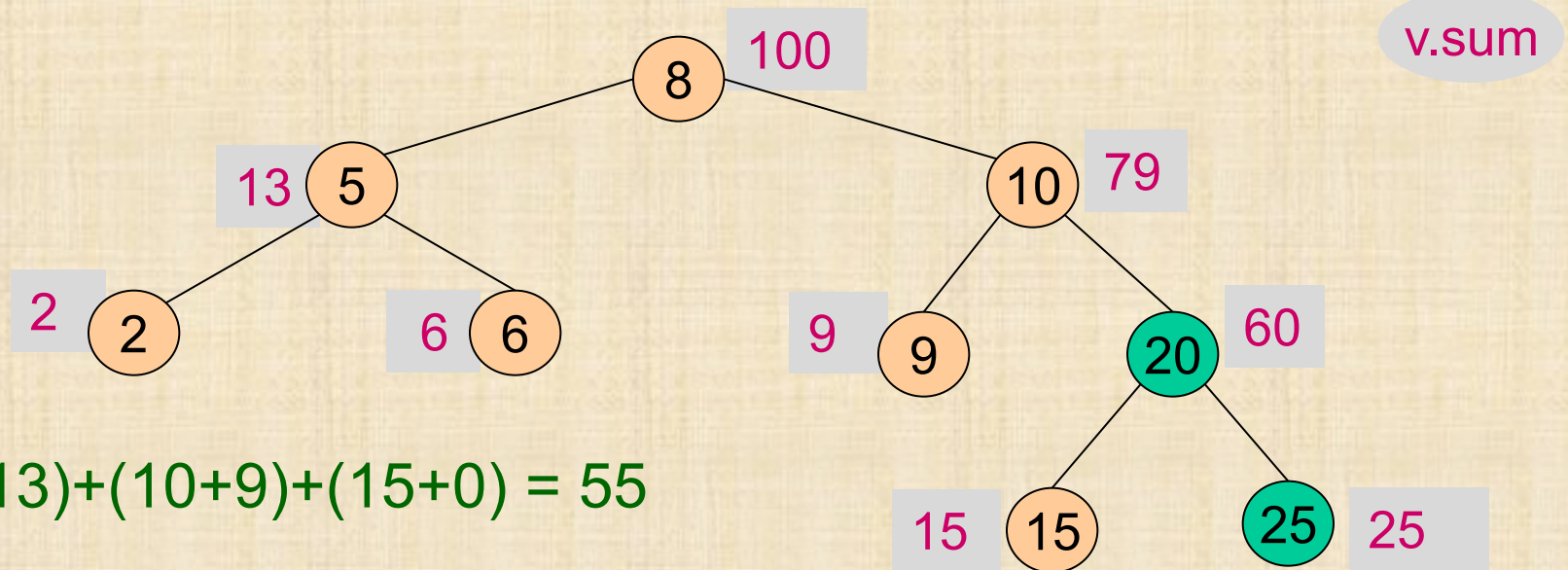
▷ Where are the keys that are < 17 ?

Where is 17 ?



Sum of Keys < x

- ▷ Where are the **keys** that are < 17?
- ▷ What is the sum of keys that are < 17?



SumSmaller

SumSmaller (x,k)

if $x == \text{null}$, then return 0

if $k == x.\text{key}$

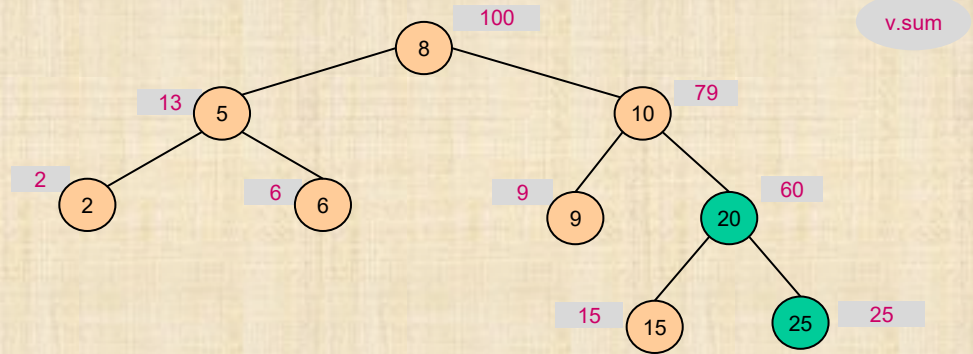
then $s \leftarrow (x.\text{left}).\text{sum}$

elseif $k < x.\text{key}$

then $s \leftarrow \text{SumSmaller}(x.\text{left}, k)$

else $s \leftarrow x.\text{key} + (x.\text{left}).\text{sum} + \text{SumSmaller}(x.\text{right}, k)$

return s



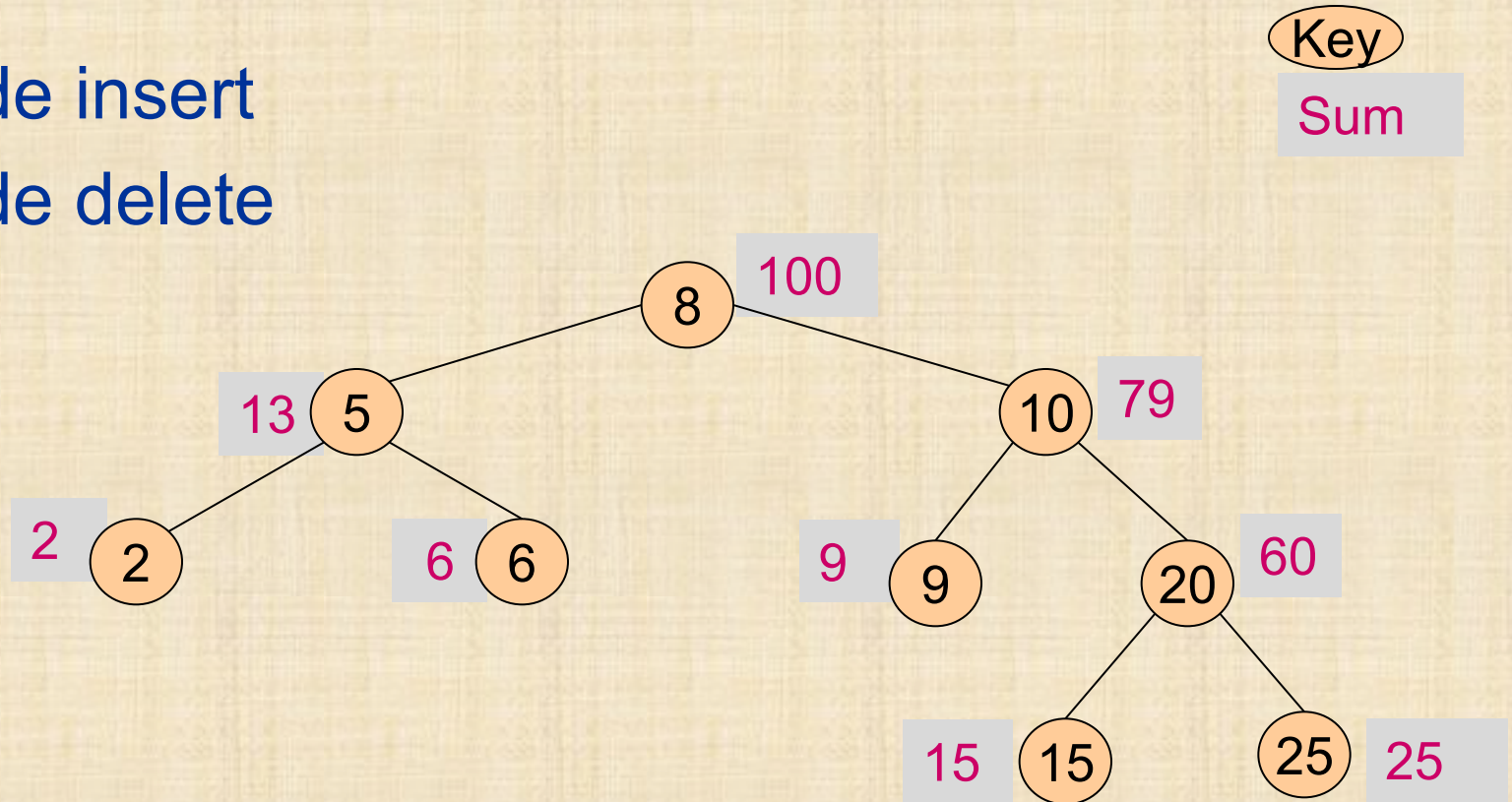
Initial call: *SumSmaller*(T,k)

Worst case running time: $\Theta(h)$

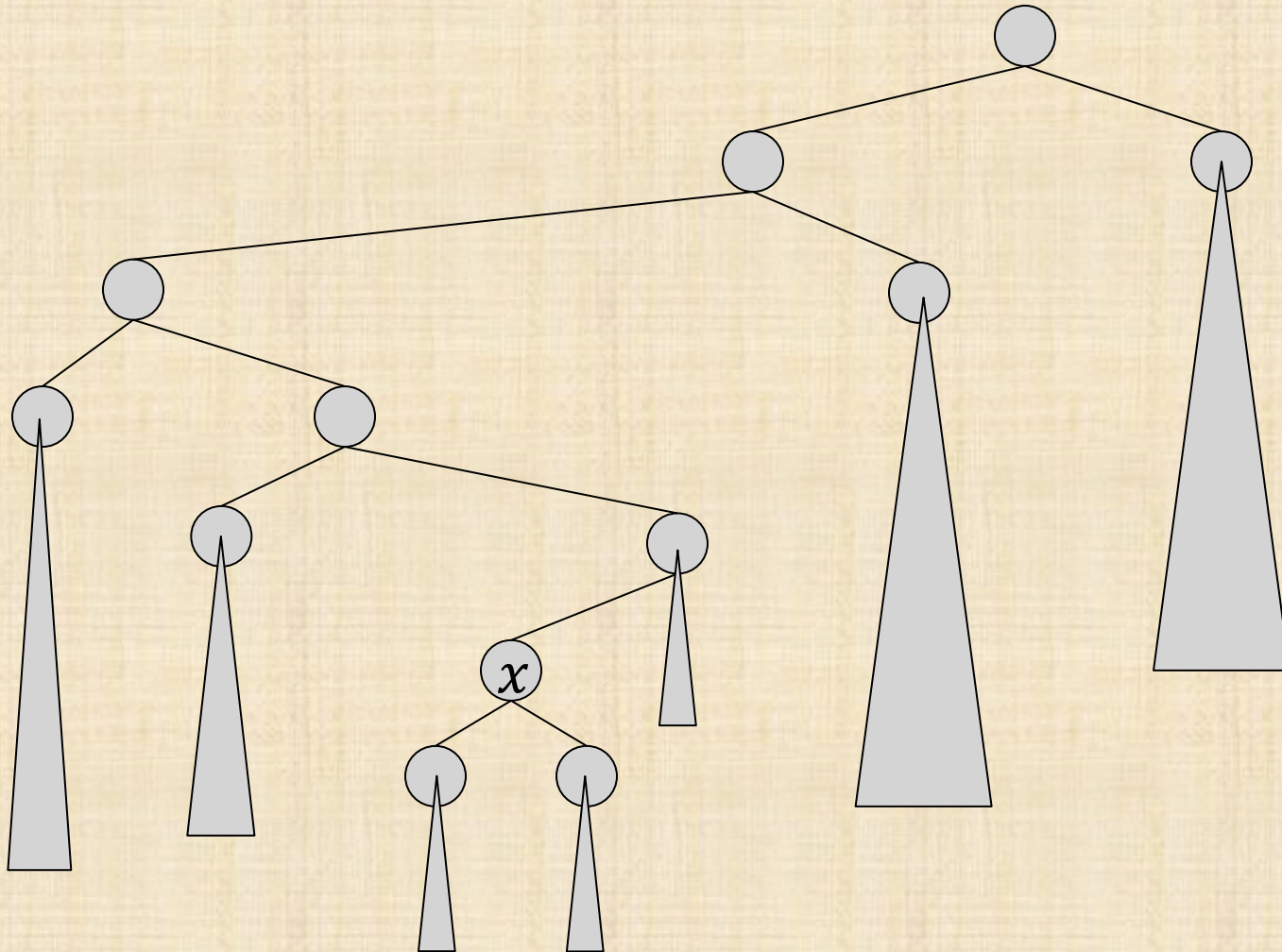
T: root of the tree
x: a node
k: a key

Updating Sum Field

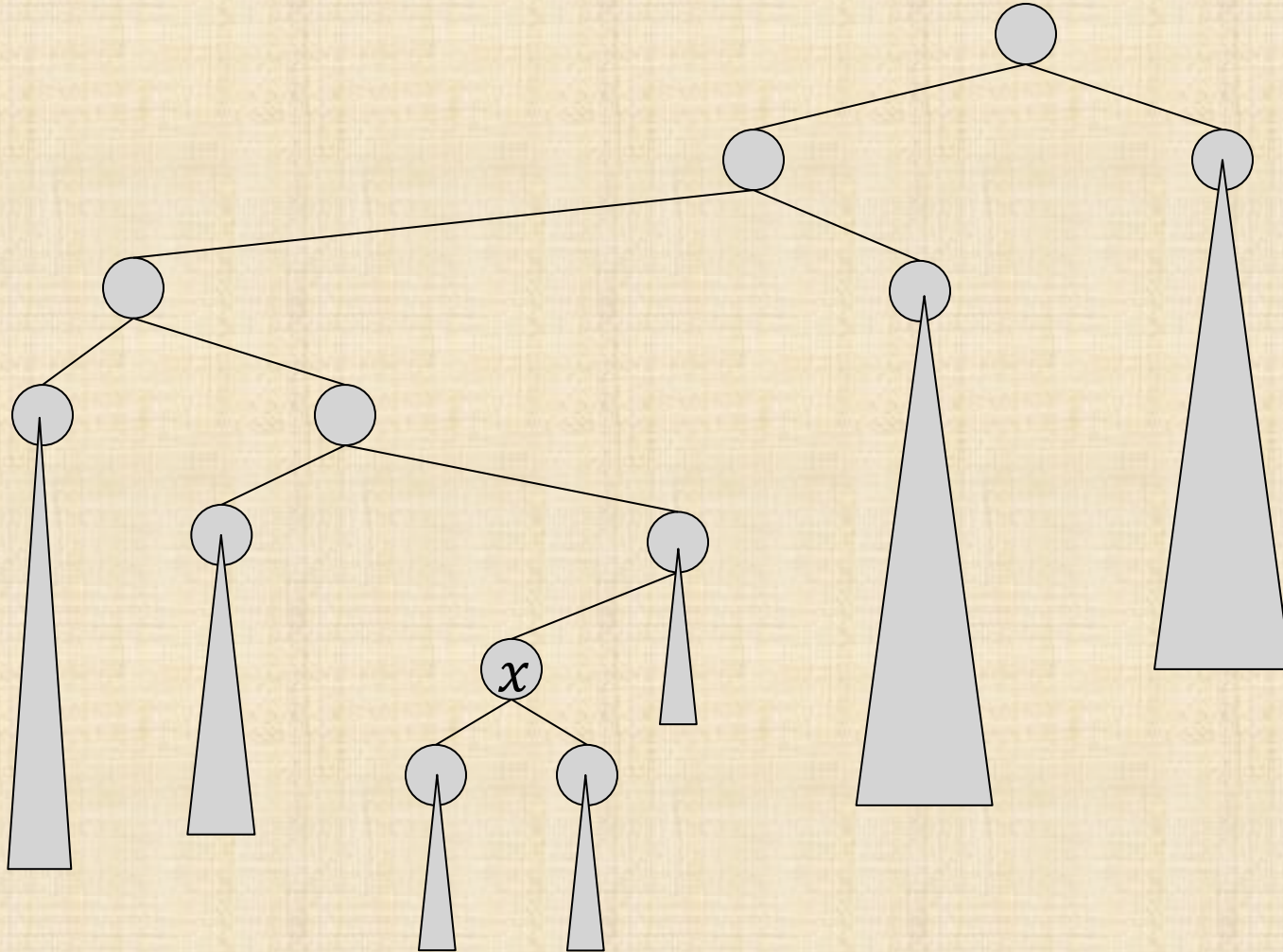
- ▷ Node insert
- ▷ Node delete



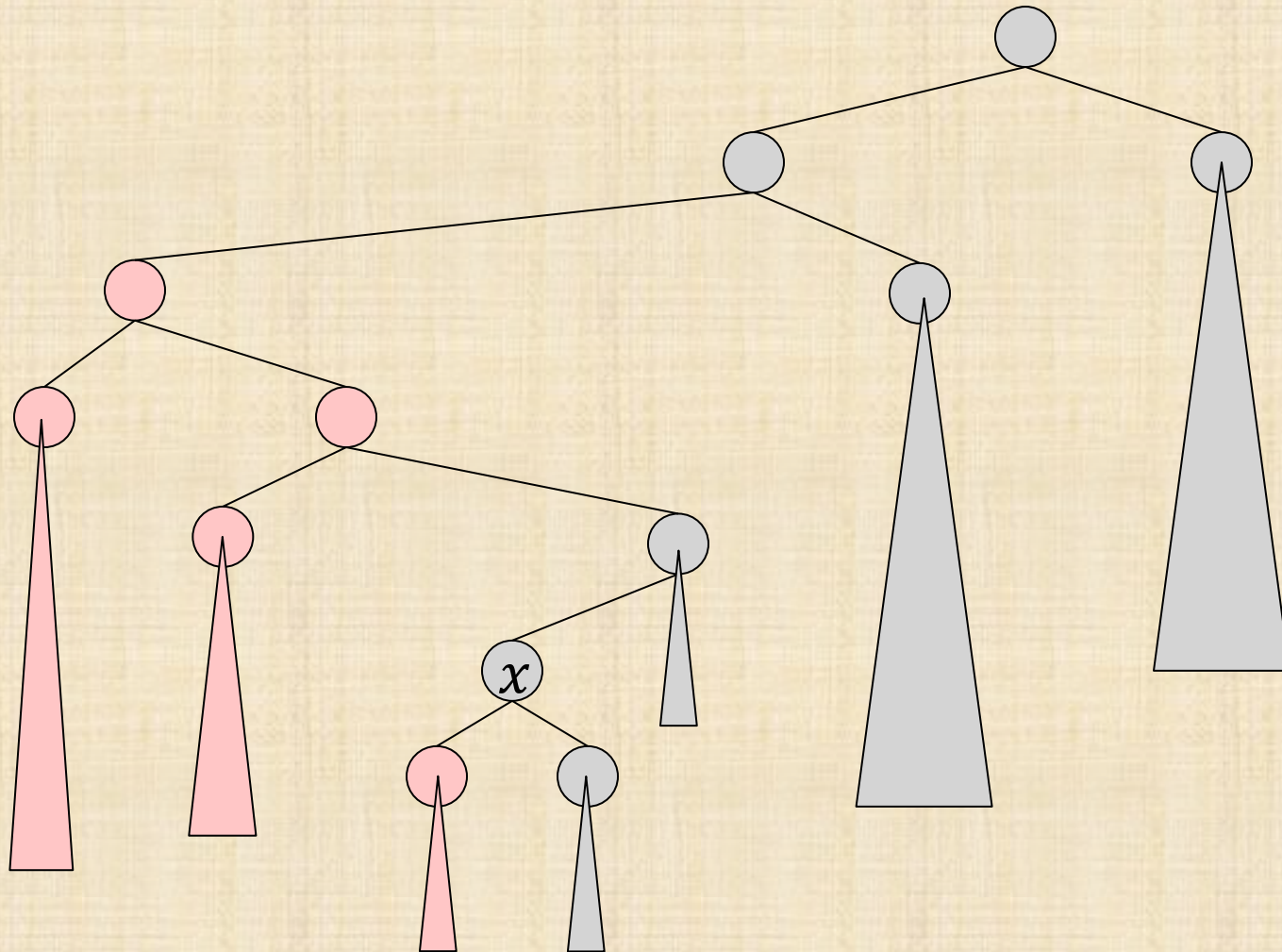
Here is a BST



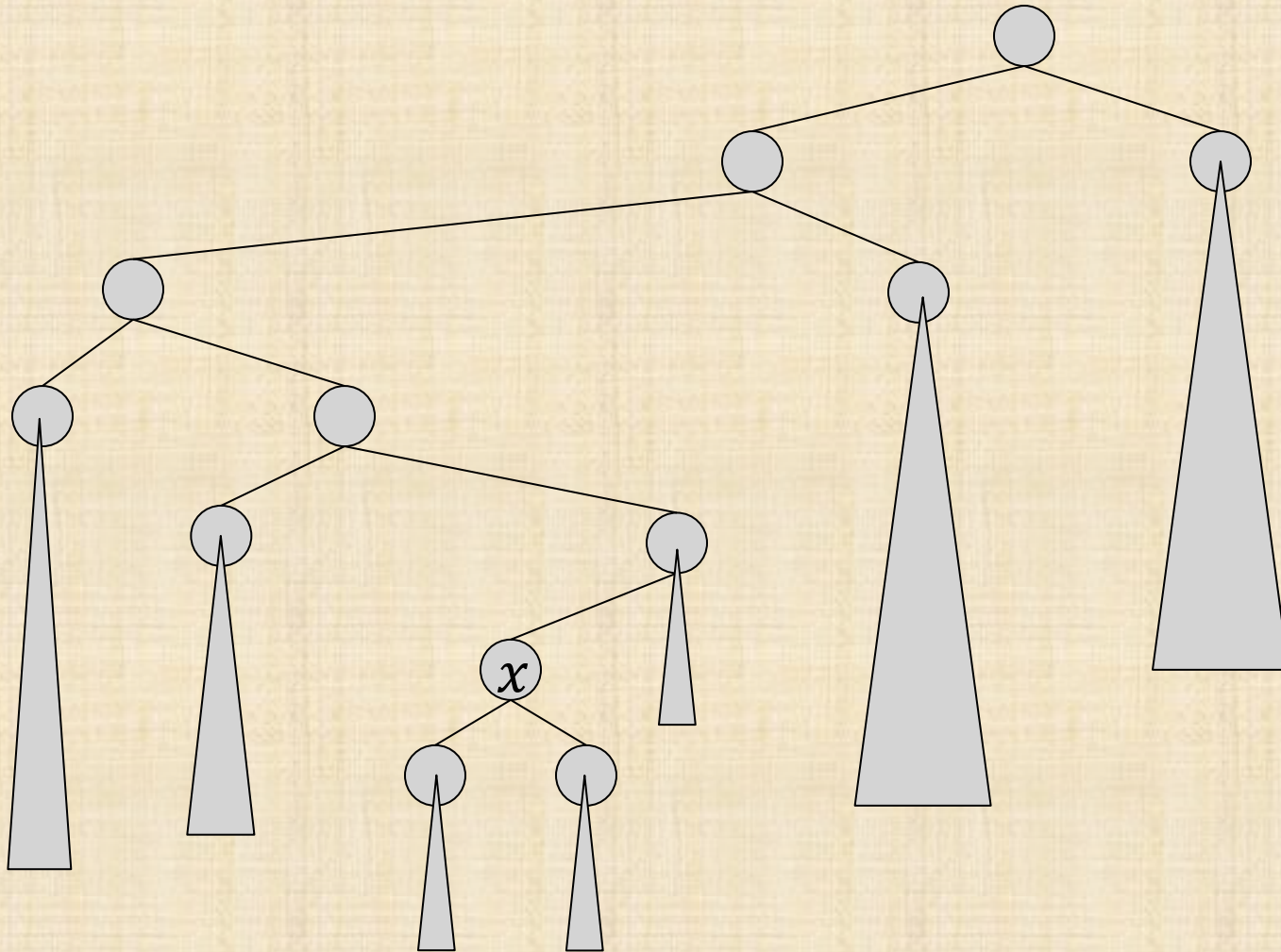
Where are the elements smaller than x



Where are the elements smaller than x



greater than x



Where are the elements greater than x

